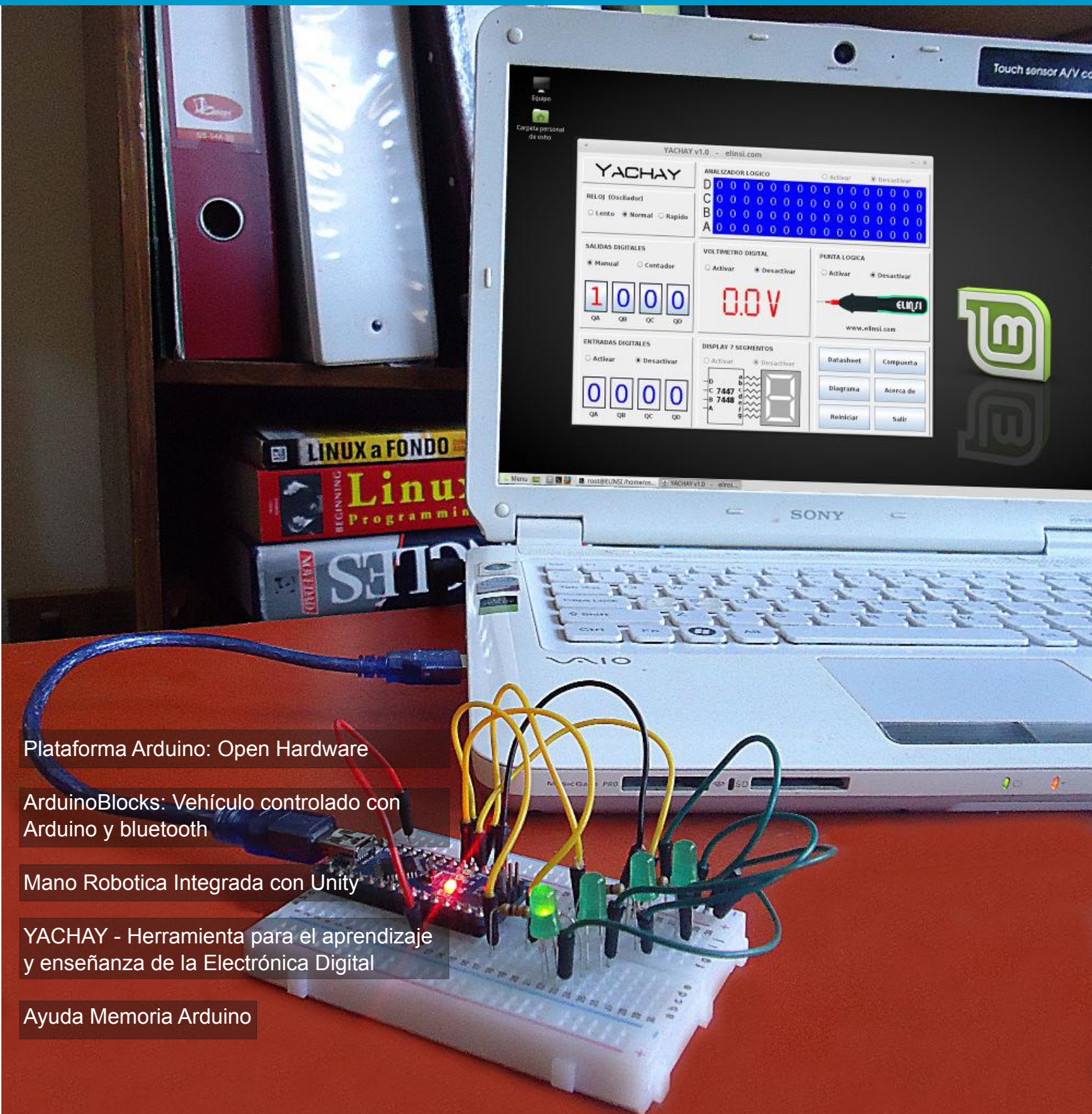


Revista Digital Arduino Bolivia

3
07/2018 - Año 0



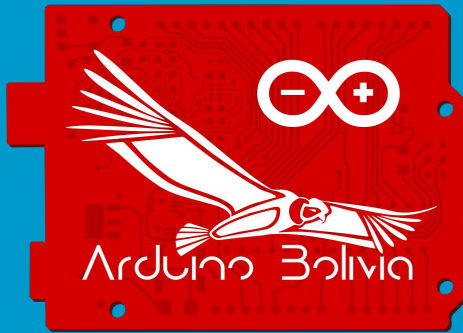
Plataforma Arduino: Open Hardware

ArduinoBlocks: Vehículo controlado con
Arduino y bluetooth

Mano Robotica Integrada con Unity

YACHAY - Herramienta para el aprendizaje
y enseñanza de la Electrónica Digital

Ayuda Memoria Arduino



 www.arduinobolivia.elinsi.com

 arduinobolivia@elinsi.com

 [RevistaArduinoBolivia](https://www.facebook.com/RevistaArduinoBolivia)

 [@Arduino_Bolivia](https://twitter.com/Arduino_Bolivia)

Esta publicación y todo su contenido se comparte con la Licencia Creative Commons 4.0



Puedes copiar, reproducir, distribuir, comunicar públicamente la obra y generar obras derivadas siempre y cuando se cite y reconozca al autor original. La distribución de las obras derivadas deberá hacerse bajo una licencia del mismo tipo. No se permite utilizar la obra con fines comerciales.

Esta publicación fue realizada con Software Libre



Scribus



GIMP



Inkscape

En una época en donde el saber cuesta, nace la revista “Arduino Bolivia”, con el fin de compartir el conocimiento a través de experiencias, y de mantener a la población del país y el mundo entero al tanto de los trabajos que se realizan en Bolivia en el área de tecnología utilizando hardware libre, un proyecto que se mantiene con el trabajo y esfuerzo desinteresado de personas que creemos en el talento humano nacional, y que convencidos de que el mismo tiene que ser mostrado nos ponemos al frente de este proyecto y hacemos que esto sea una realidad.

Sin duda alguna todo cuesta y son esos esfuerzos que se realizan sin esperar recompensa, los que nos llenan de satisfacción. Agradecer particularmente al equipo de trabajo de la Revista Arduino Bolivia; Osman Condori y Bernardo Ordoñez, por hacer que en cada edición se reflejen lo avances tecnológicos de nuestro país.

En esta edición especial celebrando el 193 aniversario de Bolivia queremos ofrecer al público, el tercer número de la revista “Arduino Bolivia”, con un contenido bastante interesante.

Personalmente darles un agradecimiento enorme a todos y cada uno de los miembros de mi familia que con su apoyo incondicional hacen que todo sacrificio valga la pena. Sobre todo, con mucho orgullo a mis principales cómplices y compañeros de vida; Mi esposa Arianne, mis hijos Jazel y Ezequiel, por comprender y aceptar que el tiempo que le dedico a mis proyectos, es tiempo que resto de compartir con ellos, pero aun así me llenan de energía con sus sonrisas. Y no quiero dejar pasar la oportunidad para agradecer inmensamente a mis padres Jhenny Flores y Abnher Rodas quienes se encargaron de mi formación como persona, quienes me enseñaron el valor y la importancia de la familia sobre todas las cosas, quienes me enseñaron a forjar mis sueños y trabajar incansablemente por conseguirlos, a mi abuelita Irene Lopez, a quien tengo la dicha de abrazar todavía, por enseñarme a no conformarme con las cosas y hacerme comprender que siempre se puede pedir más.

Jahzeel Issac Rodas Flores

Coordinadores



Osman R. Condori Guevara

osman@elinsi.com

Electrónico, Gerente propietario de la empresa de servicios y capacitación técnica en Electrónica, Informática y Sistemas "ELINSI"
www.elinsi.com



Casto Bernardo Ordoñez Callisaya

ordonezcallisayabernardo@gmail.com

Electrónico en Sistema de Control Industrial y Sistemas de Computo, Co-Fundador de la Comunidad Arduino La Paz, Propietario de EPY Electrónica Bolivia.



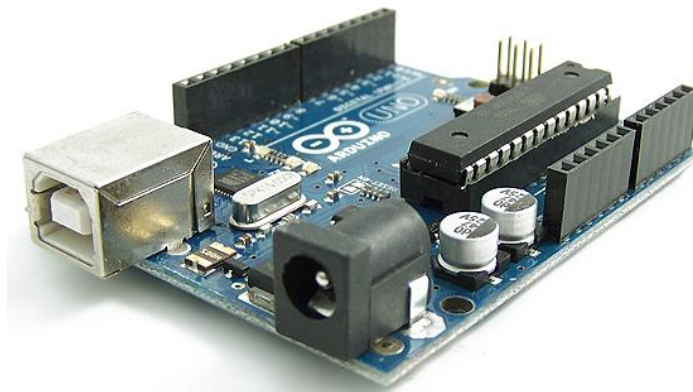
Jahzeel Issac Rodas Flores

jahzeelrodas@gmail.com

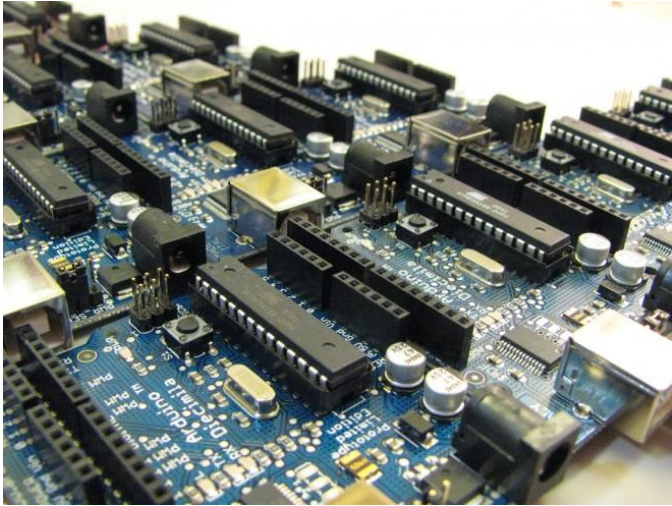
Ingeniero Informático, Experto en Robótica y Domótica con Hardware Libre, Desarrollador Web FullStack, Gerente Propietario y CEO de Robotech Tarija.

Todos los artículos, tutoriales y proyectos publicados en la revista "Arduino Bolivia" son responsabilidad de cada uno de los autores, la revista no se hace responsable de la autenticidad y posibles conflictos derivados de la autoría de los trabajos publicados.

Pag. 1	Plataforma Arduino: Open Hardware
Pag. 10	ArduinoBlocks: Vehículo controlado con Arduino y bluetooth
Pag. 17	Mano Robotica Integrada con Unity
Pag. 24	YACHAY - Herramienta para el aprendizaje y enseñanza de la Electrónica Digital
Pag. 30	Ayuda Memoria Arduino



Plataforma Arduino: Open Hardware



¿Qué es Hardware?

El Hardware es la parte tangible (que se puede tocar) de un equipo electrónico, por ejemplo, si nos referimos a una computadora, el Hardware es el Mouse, Teclado, Monitor el CPU y todo lo que lleva por dentro la Unidad Central de Proceso como ser la Motherboard, Memoria RAM, Microprocesador, etc.

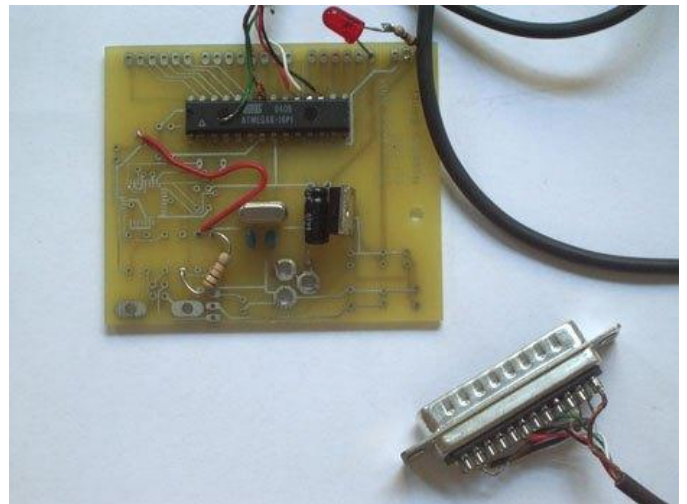


En el caso de Arduino el Hardware esta basado en nuestra placa electrónica, la cual tiene sus componentes electrónicos como ser resistencias, diodos, reguladores de tensión, capacitores, Osciladores, Leds, Micro-

controladores, puerto USB, Plug de carga, pero lo mas importante es la placa de Fibra de Vidrio sobre la que se ensamblan todos los componentes y lleva impreso el circuito electrónico.

Hardware Arduino

El hardware Arduino ha ido desarrollando varias placas para diferentes aplicaciones en el transcurso de estos 13 años, pero la placa que inicio esta revolución dio sus frutos por el año 2005 en el instituto IVREA de la mano de Massimo Banzi, esta placa estaba inicialmente basada en una simple placa de circuitos donde alojaba un microcontrolador simple con resistencias donde solo podíamos usar sensores simples.



La colaboración de los demás miembros del equipo Arduino ayudo a que esta placa pudiera ser implementada con los desarrollos en software y hardware permitiendo integrar un Bootloader y el puerto de comunicaciones USB, el cual hizo de esta placa en convertirse en el numero uno de las herramientas de aprendizaje por su versatilidad y fácil manejo.

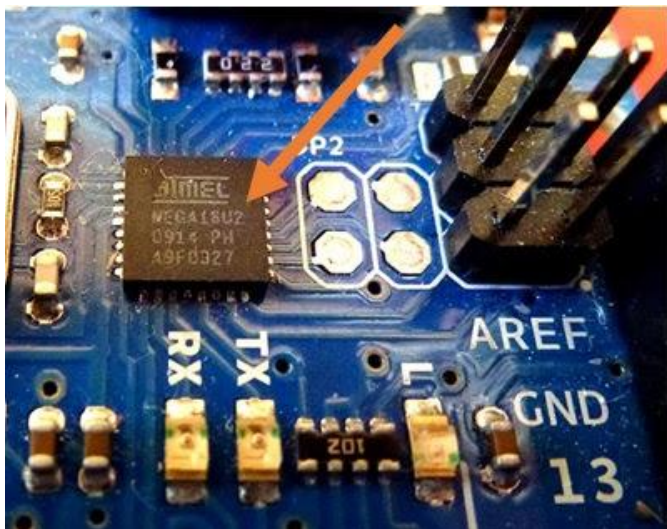
Las placas Arduino y en general los microcontroladores tienen puertos de entrada y salida, además de los puertos de comunicación, y se puede acceder a estos a través de los pines.

- Pines Digitales**, los cuales se puede usar como entradas digitales para leer sensores y como salida para escribir el control de actuadores.

- Pines Analógicos Entradas**, usan un conversor Analógico/Digital y sirven para leer sensores analógicos como de temperatura, luz, etc.

- Pines Analógicos Salidas(PWM)**, Arduino no integra un conversor Digital/Analógico para ello se utilizan salidas PWM que son pines digitales que usan la modulación de pulso, no todos los pines digitales las tienen.

- Puertos de Comunicación USB, Serie, I2C y SPI**, para establecer la comunicación (grabar placa) no se necesita de un cable FTDI, en su lugar usa un microcontrolador programado como conversor de USB a serie el Atmega16U2 que se encuentra detras del cable USB de las placas Arduino.



•Memorias del microcontrolador de las placas Arduino.

- SRAM**, donde se crea y manipula las variables cuando se ejecutan, es limitado su uso y se debe supervisar su uso.

- EEPROM**, memoria para mantener los datos después de un reset o apagado de la placa, estas tienen un numero limitado de lecturas y escrituras.

- FLASH**, Memoria del programa, desde 1Kb hasta 4Mb en donde se almacena el Sketch y el Bootloader

El Bootloader se trata de un programa especial y puede leer datos de una fuente externa como UART, I2C, CAN, etc. Se ejecuta inmediatamente antes de ejecutar el programa que hay en la memoria FLASH.

•Alimentación.

- USB**. alimentar la placa Arduino por el cable USB no demanda mucha preocupación, con esta no se puede cometer errores de polaridad ni voltaje, esta incluye un fusible para su proyección lo cual limita la circulación de corriente a 500mA.

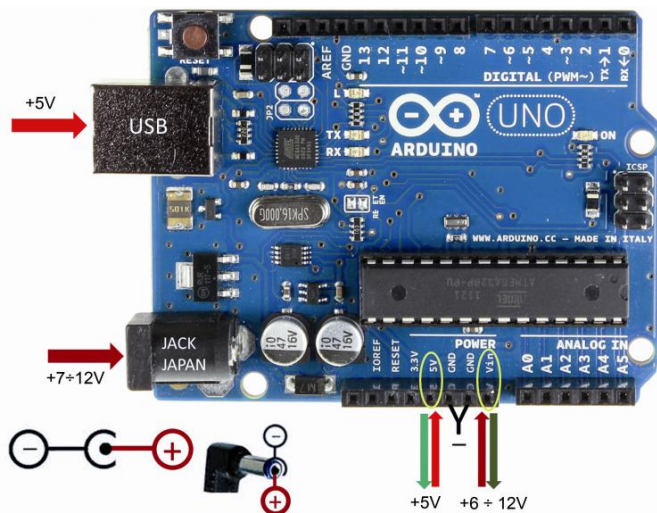
- Plug**, el conector Plug puede recibir entre 7–12V, si agregamos mayor voltaje corremos el riesgo de sobre calentar el regulador de tensión que llegaría a fundirse, esta entrada integra un diodo de protección para inversión de polaridad.

- Vin**, cumple 2 funciones:

1. Alimentación externa entre los rangos de 7-12V de manera directa a la entrada del regulador, en este caso no cuenta con protección contra inversión de polaridad. En caso de aplicar voltaje directamente al pin VIN, no se debe aplicar simultáneamente un voltaje en el Plug.

2. Salida de Voltaje cuando el Arduino se alimenta a través del Plug, podremos usar esta alimentación para otros dispositivos tomando en cuenta la caída de tensión por el diodo

(0.7V) y evitando colocar cargas mayores a los 1000mA para evitar quemar la protección.



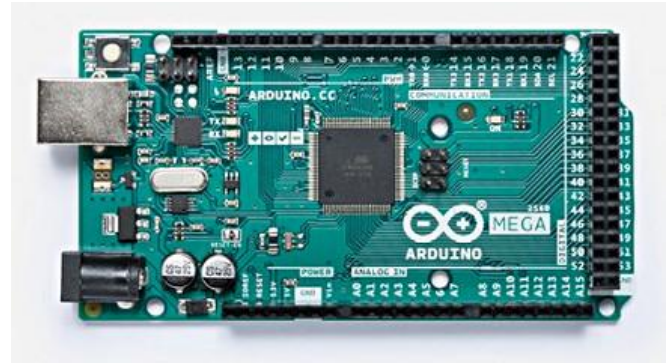
Placas Arduino

En este artículo no podremos hacer mención a todas las placas Arduino, la gran cantidad que se ha desarrollado es realmente increíble, pero más aún sus usos y aplicaciones.

NIVEL DE ENTRADA	UNO LEONARDO 101 ESPLORA MICRO NANO MINI ADAPTADOR MKR21U0
KIT DE INICIO	KIT DE INICIO
CARACTERÍSTICAS MEJORADAS	MEGA CERO DEBIDO MEGA ADK M0 M0 PRO MKR ZERO MOTOR SHIELD
	USB HOST SHIELD PROTO SHIELD MKR PROTO SHIELD 4 RELÉS SHIELD
	MEGA PROTO SHIELD MKR RELAY PROTO SHIELD MKR PUEDE ESCUDO MKR-485 SHIELD
	MKR MEM SHIELD MKR CONNECTOR CARRIER ISP USB2SERIAL MICRO
	CONVERTIDOR USB2SERIAL
INTERNET DE LAS COSAS	YÚN ETHERNET TIAN INDUSTRIAL 101 LEONARDO ETH MKR FOX 1200
	MKR WAN 1300 MKR CSM 1400 MKR WIFI 1010 UNO WIR REV2 MKR NB 1500
	MKR VIDOR 4000 MKR1000 YÚN MINI YÚN SHIELD SHIELD SD INALÁMBRICO
	PROTO SHIELD INALÁMBRICO ETHERNET SHIELD V2 GSM SHIELD V2 MKR ETH SHIELD
	MKR IOT BUNDLE
EDUCACIÓN	CTC 101 KIT DE INGENIERÍA
USABLE	CEMA LILYPAD ARDUINO USB LILYPAD ARDUINO DIRECTORIO PRINCIPAL
	LILYPAD ARDUINO SIMPLE LILYPAD ARDUINO SIMPLE SNAP

De todos modos empezaremos hablando de las placas mas utilizadas y algunas que quizá nunca viste.

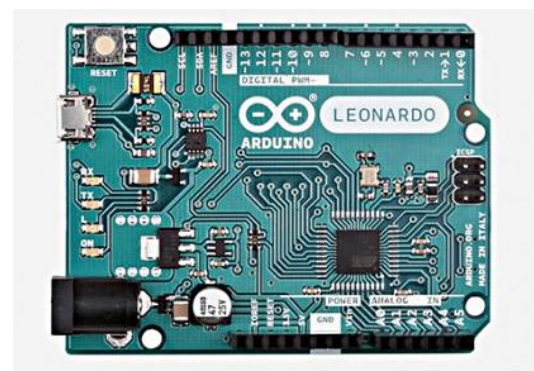
Arduino Mega



Esta es la placa que mas pines tiene para sus aplicaciones, apto para trabajos mas complejos, la base de esta placa es el Atmega2560 el cual lleva una memoria interna de 256Kb, nos brinda 54 Pines de E/S Digitales, 16 Entradas Analógicas

Mas detalles [AQUI](#)

Arduino Leonardo

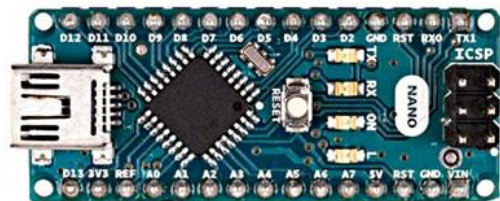


Similar al Arduino Uno pero con la gran diferencia que esta placa integra un microcontrolador Atmega32u4 el cual tiene integrado el puerto USB, lo que elimina el segundo microcontrolador, al conectarlo al ordenador requiere un nivel en esta plataforma para su manejo.

Esta placa nos ofrece 20 pines E/S Digitales de los cuales 7 son PWM y 12 entradas Analógicas.

Mas detalles [AQUI](#)

Arduino Nano



Es el Arduino Uno pero mas compacta, basado en el Atmega328P cuenta con 22 Pines E/S Digitales y 6 Pines PWM, 8 Entradas Analógicas, Funciona con un Cable Mini USB

Mas detalles [AQUI](#)

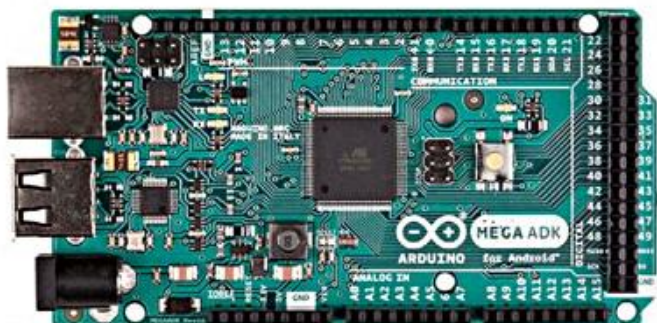
Arduino Mini



Es una pequeña placa basado en el microcontrolador Atmega328, no cuenta con una conexión USB a Serie incorporada, tiene 14 pines Digitales E/S de los cuales 6 son PWM y 8 entradas Analógicas.

Mas informacion [AQUI](#)

Arduino Mega ADK



Android colaboro en el desarrollo de una placa que permitiera una integración con el ADK.

Es así que sale a la luz una placa basado en el Mega2560 con el añadido de integrar una interfaz USB Host basado en el MAX3421e IC en cual le permite funcionar con Android.

Mas información [AQUI](#)

Arduino Zero

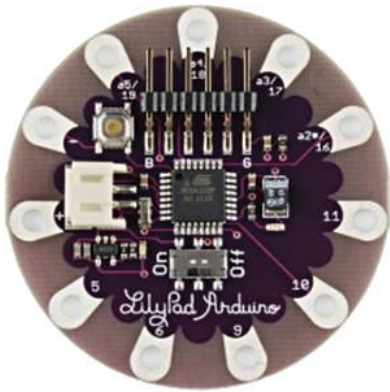


Es una extensión potente de 32 Bits de la plataforma establecida por la UNO, orientada para dispositivos inteligentes IoT, automatización y tecnología portátil y robótica. Esta placa está basada en el ATSAMD21G18, ARM Cortex M0 de 32 bits, funciona con 3.3V, 20 Pines E/S Digitales y todos son PWM menos el pin 2 y 7, Entradas analógicas 6 de ADC 12 bits, Memoria de 256Kb, todos los pines tienen interrupciones excepto el 4.

En definitiva, esta placa ha marcado un antes y un después en la plataforma Arduino, pero solo fue el principio.

Mas información [AQUI](#)

Arduino Lilypad

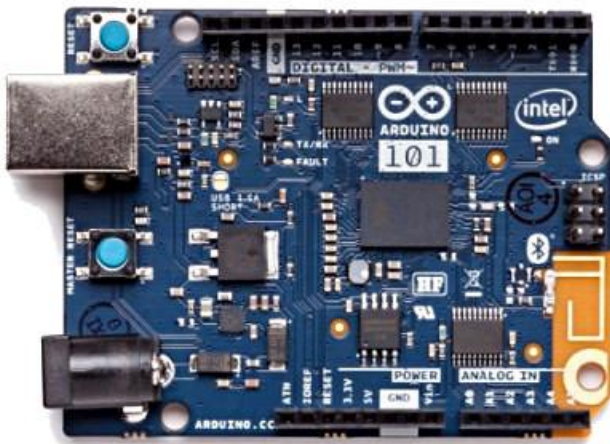


Es la placa perfecta cuando se trata de hacer un proyecto textil, se la puede cocer a la tela y a las fuentes de alimentación y sensores o actuadores con hilo conductivo, tenemos varias versiones entre ellas:

Lilypad Arduino USB, basada en el Atmega32u4 y tiene 9 pines Digitales E/S de los cuales 4 pueden usarse como PWM y 4 entradas Analógicas, funciona con 3.3V e integra un conector JST para baterías LiPo de 3.7V.

Mas información [AQUI](#)

Arduino 101

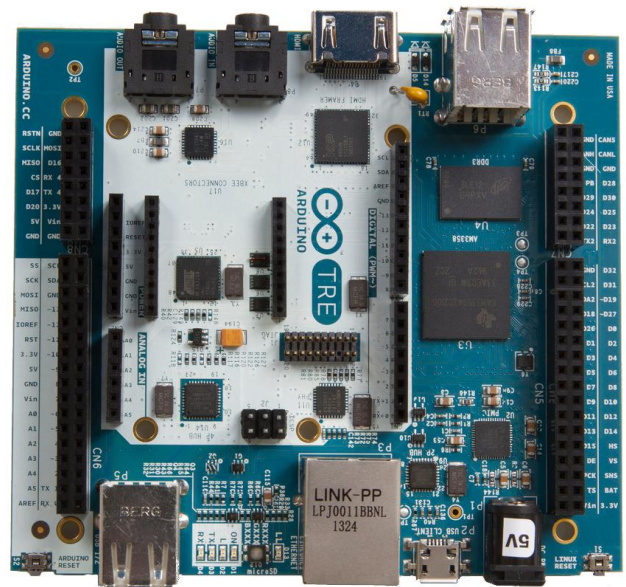


Esta placa salió a principios del 2016, manejando el clásico tamaño de la Arduino Uno, pero integrando lo último en tecnologías. Basado en el microcontrolador Intel Curie de bajo consumo de energía, incorpora un Bluetooth LE y un Acelerómetro / Giroscópico de 6 ejes, este módulo integra 2 núcleos, un 86x (Quark) y un núcleo de Arquitectura ARC de 32 bits cuenta con 14 pines E/S Digital de los cuales 4 son PWM, 6 entradas Analógicas

Estas placas han sido desarrolladas en colaboración con Intel.

Mas información [AQUI](#)

Arduino TRE



Es la primera placa Fabricada en los EEUU, esta placa corre a una velocidad de 1GHz gracias a que integra el procesador Sitara AM335x ARM Cortex-A8 y una RAM de 512DDR3, esta placa abre nuevos horizontes a aplicaciones más avanzadas de Linux, usa el microcontrolador Atmega32u4 que mantiene las mismas características del Arduino Leonardo, con la adicional que esta placa lleva 4 Puertos de Host USB y 1 puerto de

dispositivo USB , HDMI (1920x1080) entrada y salida de Audio, Ethernet 10/100, 23 Pines Digitales E/S (3.3V) 4 de ellos PWM 6 pines de entrada Analógica (más 6 multiplexados en 6 pines Digitales), tarjeta Micro SD y conector de expansión LCD.

Open Hardware

Arduino es Open-Hardware, lo cual significa que podemos acceder a la documentación y los diseños de las placas y podemos modificar o duplicar.

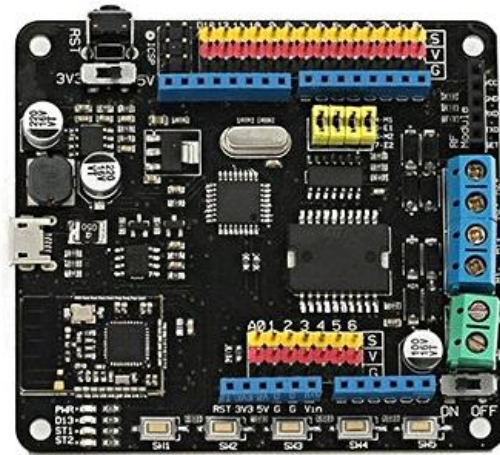
Y como el Hardware Arduino fue la fascinación de muchos, no se tardaron en salir algunas placas Arduino interesantes.

Arduino Chipkit uC32



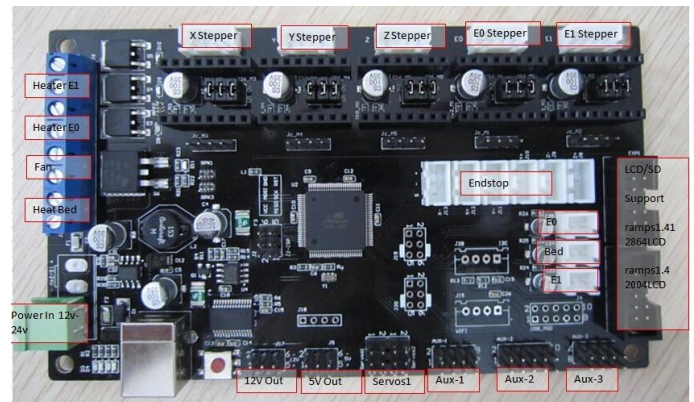
Tarjeta de desarrollo basada y hecha para ser totalmente compatible con la popular plataforma de desarrollo Arduino™, usando microcontroladores de 32 bits PIC32 de Microchip, trabaja a una frecuencia de 80Mhz, nos ofrece 42 pines E/S, 12 entradas Analógicas, 5 pines PWM, 2 Puertos UART, 1 puerto SPI, 1 puerto I2C, 1 puerto ISCP, integra un módulo RTCC y un puerto CAN.

BLE Motor



Esta placa desarrollada por Freaduino, basado en el Arduino Uno que integra el Puente H L298P y Bluetooth 4.0 en una sola placa, Podemos usarlo para conducir un motor paso a paso de una vía o motor de CC bidireccional además de agregar sensores y actuadores con la alimentación ya distribuido en los header.

Arduino MKS Gen



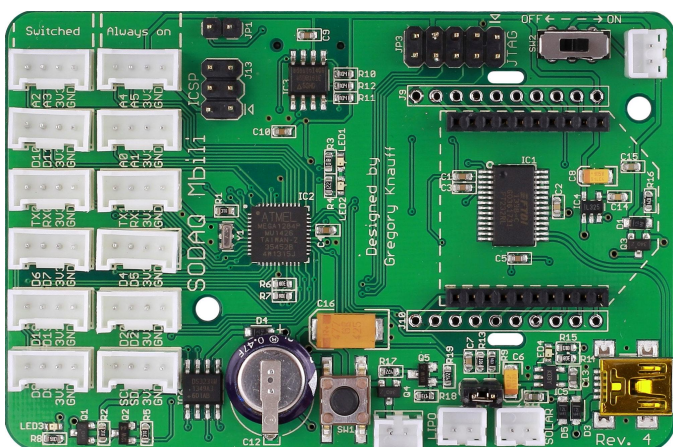
Esta placa ha sido desarrollada a partir del Arduino Mega y la RAMPS1.4 combinando ambas placas en una sola sacando provecho a la potencia y agregando todos los conectores necesarios para que esta placa funcione en una impresora 3D y se pueda agregar los Drivers, motores, sensores y pantalla LCD sin dejar de lado la distribución de energía para todo el sistema.

Rascal



Una placa totalmente compatible con las Shield de Arduino es la Rascal basado en un ARM el AT91SAM9G20, esta placa se programa en lenguaje Phyton la cual lo hace muy sencilla de programar, nos permite ejecutar Linux con puertos de Red y slot SD para el almacenamiento.

SODAQ



Desarrollada por Solar Powered Data Acquisition, nos ofrece una serie de sockets para conectar módulos Xbee, RFbee, bluetooth y GPRS, basado en el Atmel1284P tienen un puerto mini USB con indicadores de batería para LiPo y un RTC Ds3231.



Estas placas solo son algunas a las que hacemos referencia, Arduino empezó una revolución por el desarrollo en diferentes campos y para ello el desarrollo de plataformas que se adapten a cada una de estas situaciones ha dado a luz a diferentes placas, tan solo es cuestión de buscar en la red y encontraremos una placa que se ajuste a nuestros requerimientos, veras que lo encuentras.

Arduino Original o Clon ¿Que es mejor?

Puede parecer una simple pregunta, pero, todos deben saber, Arduino es una Plataforma Open Source y Open Hardware, eso significa que es de uso libre tanto el software y el hardware, esto llevo a que Arduino publicara los circuitos electrónicos de sus placas tanto el PCB y el Esquemático.



Sino me creen ingresen acá les dejo los enlaces por placas:

[Arduino Uno](#), [Arduino Mega](#), [Arduino Leonardo](#) y [Arduino Nano](#).

Con esto queremos decir que Arduino busco que la plataforma no sea solo comprar una placa oficial y empezar a desarrollar, sino que permitió que cualquiera agarre y desarrolle su propia placa, es por eso que la industria China agarro y en base al circuito pudo mejorar o abaratar costos en la manufactura de estas, es de ahí que en el mercado existen placas Arduino mas baratas que la misma oficial o también mas caras, pero mucho depende de la producción y calidad de estas, al final las placas que siguen el esquema de Arduino son totalmente compatibles con el software, solo las placas que hacen la modificación más tradicional como por ejemplo el abaratar el costo en el Chip de comunicaciones al agregar un FTDI CH340G en vez de un Atmega16U2 son razones por las que se debe instalar un software extra pero luego son funcionales, todo esto le ha permitido a Arduino ser una placa que este al alcance de todos.

Arduino Uno

Nos hemos reservado hablar del Arduino Uno hasta esta parte para que podamos entender la importancia que tiene esta placa.

La Arduino UNO esta catalogada como la MEJOR placa para comenzar a usar la electrónica y la codificación, ideal para escolares y universitarios, la práctica manera de desarrollo con esta placa ha llevado a que se desarrolle basta documentación de todo tipo de proyectos y tiene integración con shields, módulos y sistemas de todo tipo.

Esta fue la primera placa lanzada al mercado que integraba el puerto de comunicaciones USB en su Rev.1 de manera conjunta con el software Arduino (IDE) 1.0.



Original

Originalmente esta placa se fabrica en Italia, pero la producción de mayor cantidad para el resto del mundo proviene de China ya que son más aceptadas por su costo.



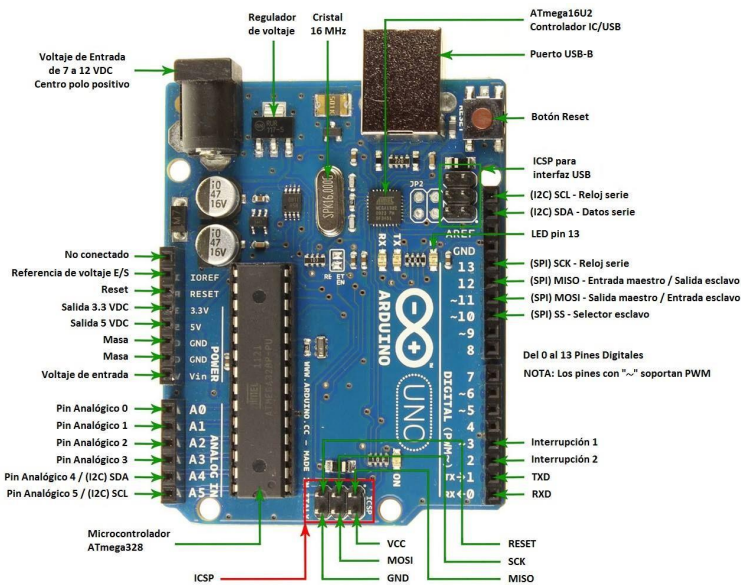
Clon

La última versión del Arduino UNO es la R3, que utiliza el Atmega328P y para la comunicación un Atmega16U2 el cual nos permite ratios de transferencia mas rápidos, no necesita de drivers, el software en su instalación lo pone a disposición cuando se conecta la placa a la computadora.

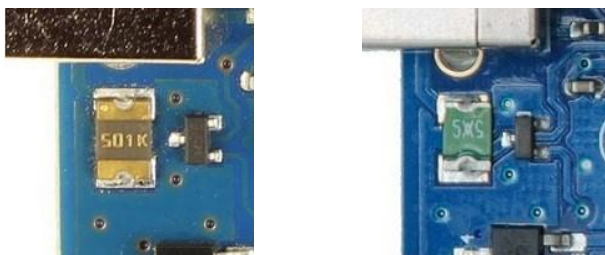
La R3 añade pin SDA y SCL cercanos al AREF, también existes 2 pines cerca del Reset, uno es el IOREF, que permite a las Shield adaptarse al voltaje brindado y un segundo pin reservado para propósitos futuros.

El Arduino Uno nos ofrece 14 pines E/S Digital 6 de ellos PWM y 6 entradas Analógicas, estos pines pueden trabajar con una intensidad de hasta 40mA. Posee una memoria de 32Kb (0,5Kb Bootloader) 2K SRAM y 1 Kb de EEPROM, lleva integrado un oscilador de cuarzo de 16Mhz.

Mas información [AQUI](#)



¿Entonces cual comprar?



Original

Clon

Ahora si vamos a decir cual es mejor, yo les diría que una Placa Arduino Oficial.

Porque ha sido desarrollada con componentes electrónicos de calidad que cumplan los estándares que exige cada placa, el quemado de placa es de primera y ni hablar de la serigrafía son de calidad, pero no solo queda ahí, al comprar una placa Arduino Oficial

contribuyes a que esta plataforma siga desarrollando nuevas placas, tengamos actualizaciones de software y la comunidad siga creciendo.



Original



Clon

Si tu quieres que no muera Arduino te invito apoyar adquiriendo una placa Oficial o haciendo un aporte voluntario.

Para encontrar distribuidores oficiales en tu País ingresa [AQUI](#) Donaciones [AQUI](#)

Elaborado por:



Mi nombre es [Casto Bernardo Ordoñez Callisaya](#), Nacido en La Paz – Bolivia, estudiante de último grado en [EISPD](#) a nivel Técnico Sup. en la Carrera de Electrónica en Sistemas de Control Industrial, Propietario de [EPY Electrónica Bolivia](#), Co-Fundador de La [Comunidad Arduino La Paz – Bolivia](#)

ArduinoBlocks: Vehículo controlado con Arduino y bluetooth

En este tutorial se presenta un vehículo controlado por bluetooth desde un teléfono móvil. Se programa con un lenguaje gráfico por bloques en la plataforma www.arduinoblocks.com, por lo que resulta un proceso sencillo, intuitivo y fácilmente comprensible.

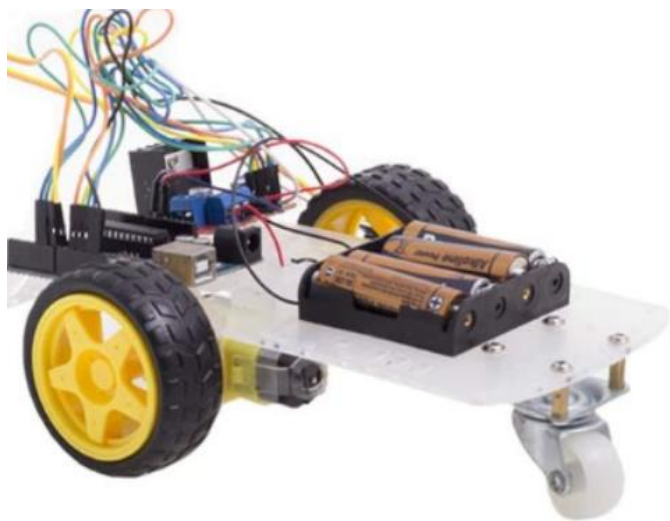


Figura 1: Vehículo controlado por bluetooth

Una de las razones por las que la programación resulta tan atractiva y dinámica es la posibilidad de programar de diferentes formas una misma tarea. No obstante, esto ha generado, en ocasiones, una complejidad innecesaria de los algoritmos diseñados. Como al contar una historia o al explicar un concepto nuevo, la programación debe ser clara y sencilla. Y esto puede ser, en realidad, lo más complicado, ya que requiere una comprensión más profunda de todo el proceso. En palabras de Einstein, *“si no lo puedes explicar de forma sencilla, es que no lo has entendido bien”*.

El paradigma de programación gráfica propuesto, junto con la utilización de bloques

de funciones ya predefinidos en ArduinoBlocks, hace este proceso mucho más accesible, sobre todo para usuarios menos expertos. Para usuarios con más experiencia, teniendo en cuenta el desarrollo actual de la plataforma y la multitud de bloques y librerías disponibles, ofrece una nueva forma de programación rápida y visual. Además, muestra una visión clara de la estructura del programa y de las estrategias seguidas en el mismo.

Vamos a trabajar con el kit de la figura 1, formado por dos ruedas motrices y una rueda giratoria central, pero en la parte final de este artículo, se presentan también otras alternativas para la construcción del vehículo, junto con la posibilidad de fabricar uno propio. Además, todas estas propuestas pueden ser completadas con sensores que otorguen otras funcionalidades añadidas como las de «sigue línea» o «esquiva obstáculos».

Partes que componen el vehículo y esquema de conexión

Para realizar los giros no se utilizan servomotores ni otras partes móviles en su sistema de dirección. Estos cambios de dirección se llevan a cabo haciendo girar las ruedas a diferentes velocidades. Si la rueda (o ruedas) del lado derecho giran a mayor velocidad que las del lado izquierdo, el coche gira a la izquierda, y viceversa.

Esta característica simplifica la fabricación y dota de mayor capacidad de giro al vehículo; sin embargo, obliga a usar al menos dos motores.

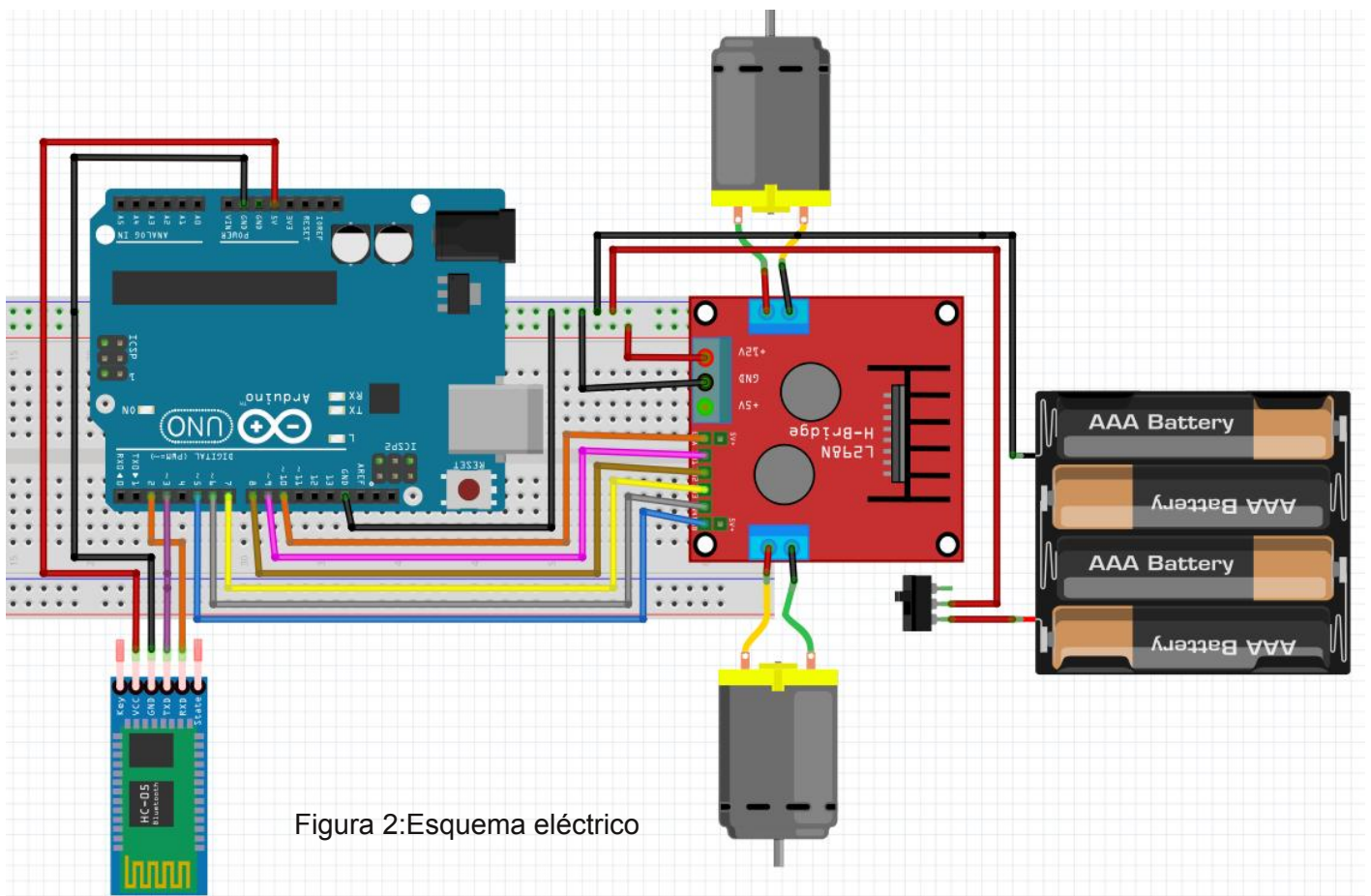


Figura 2:Esquema eléctrico

1.Arduino UNO.

También se puede programar desde ArduinoBlocks el Arduino Nano, lo que permite realizar vehículos más pequeños y ligeros (como el velocista que veremos al final del artículo). Igualmente, se puede usar el Arduino Mega para robots con más funciones que hagan necesario el uso de más entradas/salidas.

2.Controlador de motores L298N.

Es el elemento que se encarga de llevar a cabo el cambio de sentido de giro y la variación de velocidad de los motores.

IMPORANTE: Este elemento (o uno similar) es imprescindible. Para estas funciones no podemos usar directamente Arduino porque no tiene potencia suficiente.

3.Dos motores (o cuatro para un vehículo de 4 ruedas).

Se usan motores low cost con reductora

incluida. Son kits baratos y fáciles de usar e instalar, aunque su potencia es baja. Sus tensiones de funcionamiento oscilan desde los 0 voltios hasta los 6.

4.Rueda giratoria o rodamiento (ball caster).

5.Teléfono móvil con la app Bluetooth Electronics instalada.

Se elige esta aplicación por su versatilidad y el gran número de funciones que integra. Hay otras opciones que pueden ser más sencillas para este proyecto concreto, pero se considera que Bluetooth Electronics puede ser una herramienta interesante para muchos otros proyectos.

6. Cualquiera de los módulos bluetooth HC-05, HC-06, HC-08 o HC-10.

Los módulos HC-08 y HC-10 son BLE (Bluetooth Low Energy) y algunos teléfonos móviles Android no los reconocen. Por otro

lado, los módulos HC-05 y HC-06 (más antiguos que los anteriores) no son reconocidos por muchos móviles Apple. Uno de los puntos fuertes de Bluetooth electronics es que aumenta la compatibilidad entre módulos bluetooth y móviles.

7. Pilas o baterías que den en conjunto un máximo de 6 voltios de tensión.

Teniendo en cuenta la tensión de funcionamiento máxima de los motores utilizados, una combinación interesante para la alimentación puede ser un conjunto de 4 pilas (o baterías recargables) de 1,5 V cada una.

Programación en lenguaje gráfico por bloques con ArduinoBlocks

A lo largo de este apartado se va incluir también la explicación de la configuración de la app móvil, ya que ambas están vinculadas y son imprescindibles para comprender bien el resultado final.

La aplicación móvil elegida permite utilizar multitud de iconos diferentes. Se han elegido los siguientes:

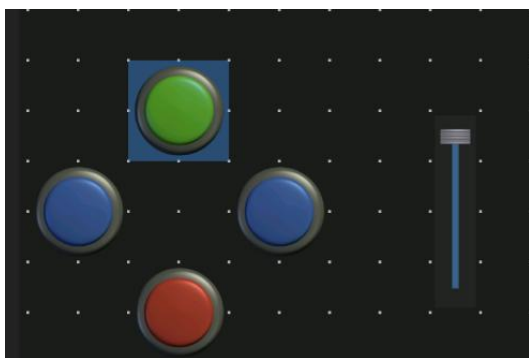


Figura 3: Botonera realizada en la app móvil Bluetooth Electronics

La velocidad la controlaremos con una barra como la que se aprecia en la parte derecha de la figura 3.

Para la dirección vamos a utilizar 4 pulsadores. Cuando pulsemos el botón verde, el vehículo comenzará a moverse hacia delante a la velocidad que indiquemos con la barra de velocidad. Mientras presionemos uno de los pulsadores azules, el vehículo girará en el sentido del lado del pulsador accionado. Al soltarlo, el vehículo volverá a correr hacia delante automáticamente, sin necesidad de pulsar de nuevo el botón verde. El vehículo se detendrá al pulsar el botón rojo, o bien cuando se reduzca a cero la barra de velocidad.

La velocidad puede cambiarse en cualquier momento, incluso cuando se está realizando un giro.

Realización del programa

En primer lugar, hay que indicar en qué pines de Arduino hemos conectado el módulo bluetooth y en cuáles el controlador de motores. Esto se conoce como inicialización de parámetros, y se incluye en Arduinoblocks dentro del bloque general Inicializar. Equivale en lenguaje de código al void setup.

Siguiendo el esquema de conexión anterior, nos quedará para el controlador de motores:

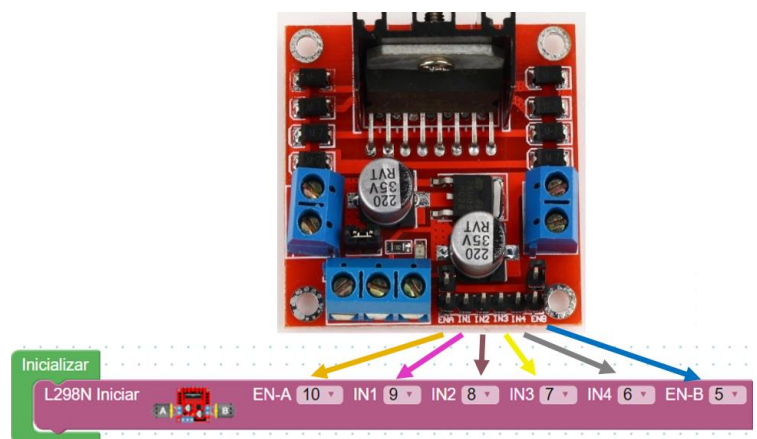


Figura 4: Configuración del bloque de controlador de motores L298N

Simplemente hay que asignar a cada entrada del controlador de motores los pines de Arduino a los que se conecta. Se pueden elegir muchas otras combinaciones diferentes a las propuestas aquí, pero habrá que tenerlas en cuenta al cablear el sistema.

Es importante destacar que hay que quitar los puentes (jumpers) que vienen de serie en ENA y ENB, con lo que queda conectado como se ve en la figura 2.

El bloque completo añadiendo el bluetooth es el siguiente:



Figura 5: Inicialización de parámetros

Una vez realizada esta primera parte de inicialización del programa, vamos a configurar el control del vehículo. Para ello nos situaremos ya en el bloque general Bucle (equivale al void loop en programación por código).

Primero, hay que comprobar si la comunicación por bluetooth es correcta (figura 6). Dentro de esta condición incluiremos el resto del programa, que sólo se ejecutará, por tanto, si tenemos comunicación.

A continuación, si la comunicación es correcta, vamos a almacenar los datos que recibamos por bluetooth en una variable. Esto consiste simplemente en darle a los datos recibidos un nombre con el que poder trabajar.

Estos dos primeros puntos se muestran a continuación:

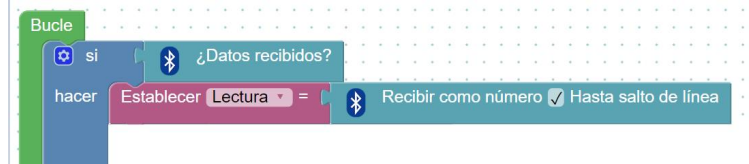


Figura 6: Lectura de los datos recibidos por bluetooth.

Las instrucciones (datos) que se envían por bluetooth desde la app móvil, pueden ser de dos tipos:

1. Cambio de velocidad del vehículo
2. Cambio de dirección del vehículo.

Debido a las propias características de la placa Arduino UNO, la variación de la velocidad se corresponde con el rango de valores comprendido entre el 0 (el motor no gira) y el 255 (máxima velocidad). Por tanto, para controlar la velocidad enviaremos números entre el 0 y el 255 desde la app móvil. La configuración de esta función en la app móvil la podemos observar en la figura 7.

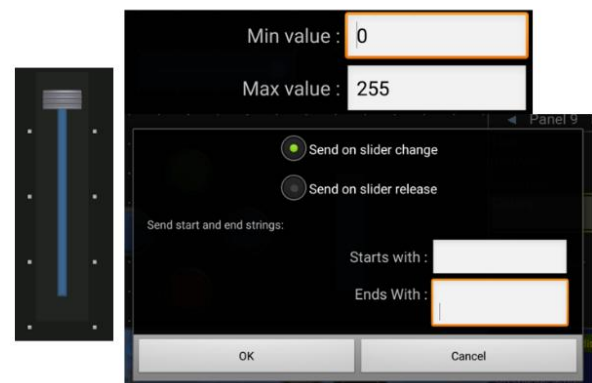


Figura 7: Configuración de la barra de velocidad en la app móvil.

En cuanto a la dirección, su control se basa en la asignación de valores superiores a 255 para cada uno de los movimientos posibles. De esta forma, cada vez que Arduino reciba un valor igual o inferior a 255, sabrá que es una orden de cambio de velocidad. En cambio, cuando reciba un valor superior a 255 previamente establecido, sabrá que es un cambio de dirección.

Las dos condiciones que clasifican las órdenes en cambios de dirección y en cambio de velocidad, según lo explicado, son estas:

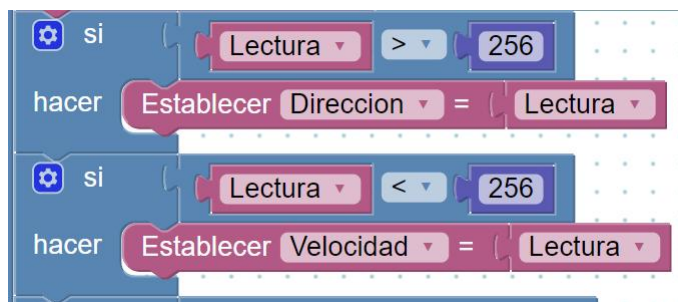


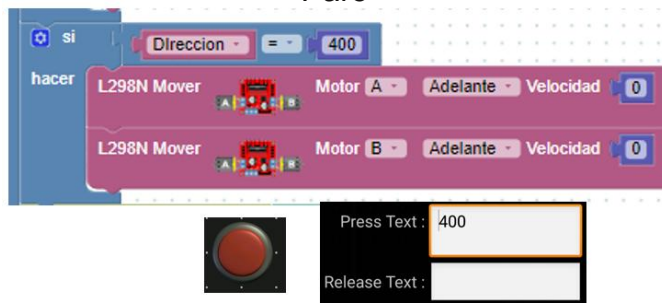
Figura 8: Condiciones para diferenciar entre distintos tipos de órdenes

Finalmente vamos a ver cómo gestionar la marcha del vehículo. Se incluye en cada imagen también la configuración de la app móvil, ya que es importante para entender el programa.

Vehículo hacia delante



Paro



Como vemos, para que el vehículo vaya hacia delante en línea recta tenemos que ordenar a ambos motores (cada uno a una rueda), que giren a la misma velocidad. En este caso se ha hecho con un 500 para marcha adelante y 400 para paro.

Giro en un sentido



Giro en sentido contrario al anterior



Para que se gire en un sentido, como decíamos, hacemos que una de las ruedas gire más despacio que la otra, o incluso que se pare. Es en este punto donde podemos configurar la reacción del vehículo ante los giros (que sea más o menos “nervioso”).

Cuando dejemos de accionar cualquiera de los dos pulsadores de giro, justo al soltar (Release text), se envía un 500, que es la orden de marcha hacia delante. Esta opción, como casi todo, no es obligatoria, pero hace la conducción más sencilla y divertida.

Las elecciones de los números de control del vehículo (500, 400, 350 y 300) son totalmente arbitrarias. Lo único importante es que estos números sean superiores a 255.

Finalmente, destacar que es aconsejable y en muchos casos imprescindible, incluir un salto de línea (Intro) tras cada orden. Con ello se consigue que la velocidad de reacción a las ordenes sea inmediata, evitando un pequeño lag de en torno a un segundo. Este salto de línea se puede apreciar en el espacio que queda debajo de cada orden enviada en las

imágenes superiores. Lo vemos con detalle aquí:

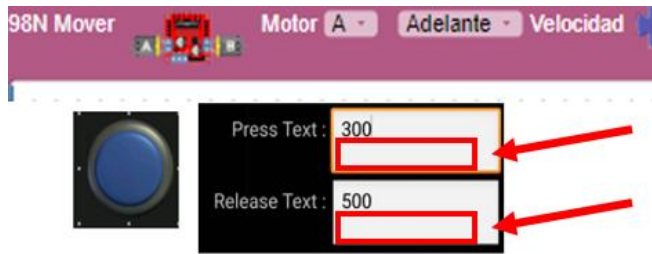
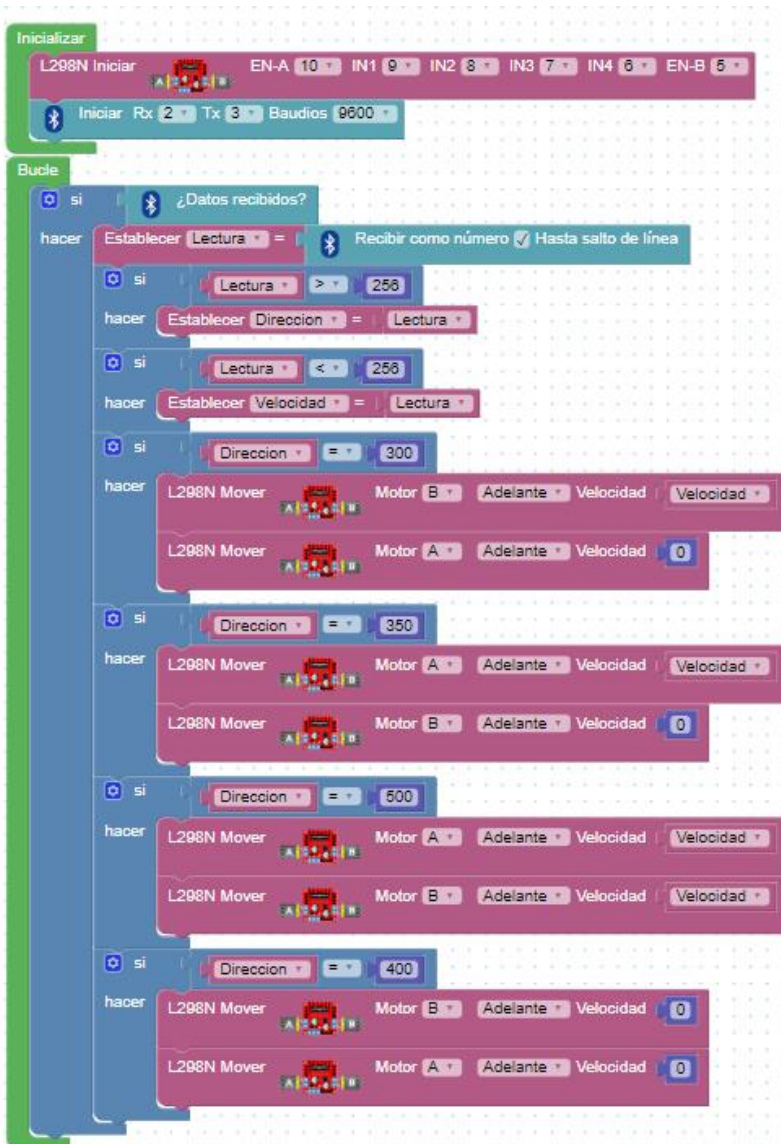


Figura 9: Salto de línea tras cada orden

Programa completo

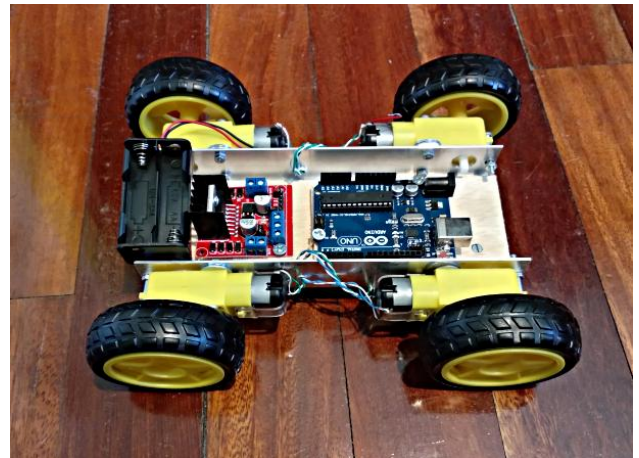


En el siguiente enlace del canal de youtube de la plataforma Didactrónica puedes encontrar un vídeo explicando todo el proceso, incluyendo la personalización de la aplicación móvil: <https://youtu.be/14hRwU1NYMo>.

También hay disponibles de forma gratuita muchos otros vídeos detallando cuestiones relacionadas, junto con otros proyectos con Arduino.

Opciones por configuración y tipo de chasis

-Versión económica con tracción a las cuatro ruedas -> Fabricación propia del chasis

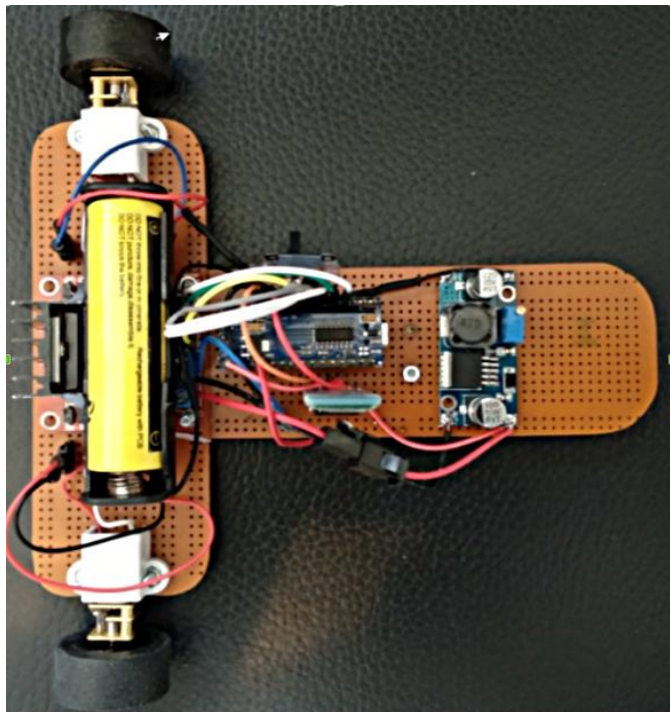


Para realizar un vehículo con cuatro ruedas y tracción en todas ellas, simplemente hay que conectar los dos motores de cada lado a la misma salida del controlador de motores.

Es importante señalar que el consumo máximo de cada uno de los motores utilizados es de un amperio y el controlador de motores admite hasta 2 amperios por cada canal. Por tanto, si aumentamos la potencia de los motores, nos veríamos obligados a usar otra configuración u otro controlador más potente. Esta cuestión se detalla en el siguiente vídeo:

<https://youtu.be/pl6OwdqvEMg>

-Versión velocista de tres ruedas -> Fabricación propia del chasis



En el blog de ArduinoBlocks también hay un artículo profundizando en los materiales y el diseño del vehículo:

<http://www.arduinoblocks.com/blog/2018/06/17/coche-velocista-con-bluetooth-y-arduino-componentes-construccion-y-programacion-en-arduinoblocks/>

En el siguiente enlace puedes acceder a la lista de reproducción de youtube de la plataforma Didactrónica en la que se explica todo el proceso de construcción y programación:

<https://www.youtube.com/watch?v=b29hfRzB14w&list=PLGMZwZq6OI909kexSSs45MHhOgWa6QQG>

Elaborado por:

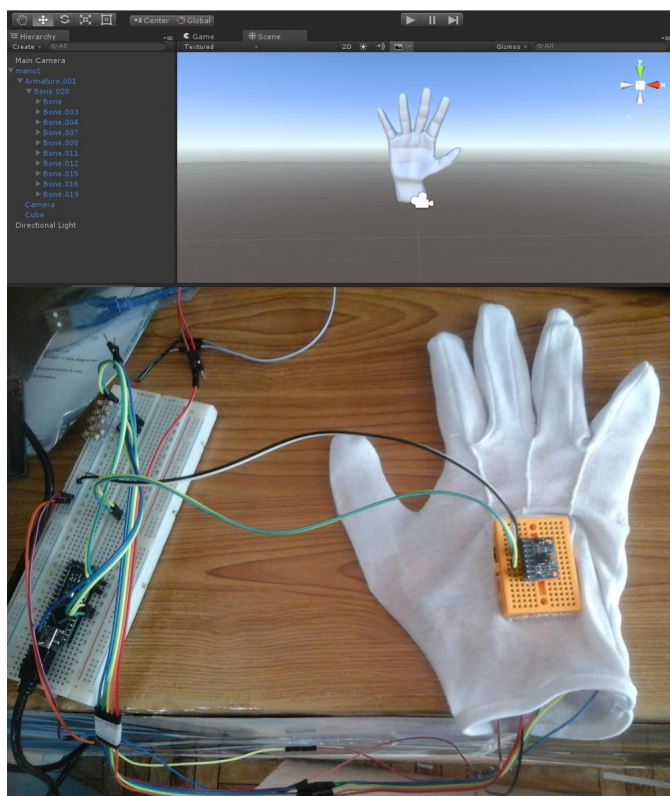


Pedro D. Domingo Fernández

Graduado en Ingeniería Eléctrica, máster en Ingeniería Electrónica y Automática, y técnico en mantenimiento industrial, es profesor de Sistemas Eléctricos y de Instalaciones Domóticas y Automáticas en Formación Profesional en el centro educativo Salesianos San José en Salamanca (España). Actualmente colabora con ArduinoBlocks y dirige la plataforma educativa con hardware libre Didactrónica.

Mano Robotica Integrada con Unity

El proyecto es un prototipo para simular gestos de la mano y dedos, interactuando con una mano virtual bajo la plataforma Unity, logrando esto utilizando sensores flex caseros un giroscopio acelerómetro MPU6050, un arduino nano y el respectivo montaje.



Antecedentes

En estos últimos años se introduce objetos virtuales en nuestra realidad, con lo que tenemos un mundo virtual a nuestro alrededor que podemos ver gracias a nuestros smartphones, tablets y gafas de RV. Pero no sólo podemos ver este mundo virtual, también podemos interactuar con él.

La realidad virtual está cada vez más presente en nuestras vidas y se encuentra en constante evolución. Para optimizar la experiencia en la

VR, un equipo de científicos de la Universidad Purdue en Estados Unidos ha desarrollado DeepHand, un sistema que captura los movimientos de la mano.

El investigador Karthik Ramani considera que las manos tienen un papel fundamental para que la realidad virtual sea completamente inmersiva. "Si tus manos no pueden interactuar con el mundo virtual, no se puede hacer nada", explica el experto.

Por eso han ideado DeepHand, que tiene la capacidad de registrar y reproducir los gestos de manera precisa. Está compuesta por una cámara con sensor de profundidad que lee la posición y el ángulo de diferentes puntos de las manos.

Después de registrar las imágenes, el software las analiza utilizando un algoritmo de aprendizaje profundo, que le permite comprender e imitar miles de posibles posiciones de las manos humanas. Funciona como una red neuronal convolucional, que es un tipo de red neuronal donde las neuronas corresponden a campos receptivos de una manera similar a las neuronas de la corteza visual primaria de nuestro cerebro.

Problemática

El problema principal es el de interactuar en el mundo virtual a través de arduino conjuntamente con los sensores que nos permiten simular movimientos y sentidos de un ser humano.

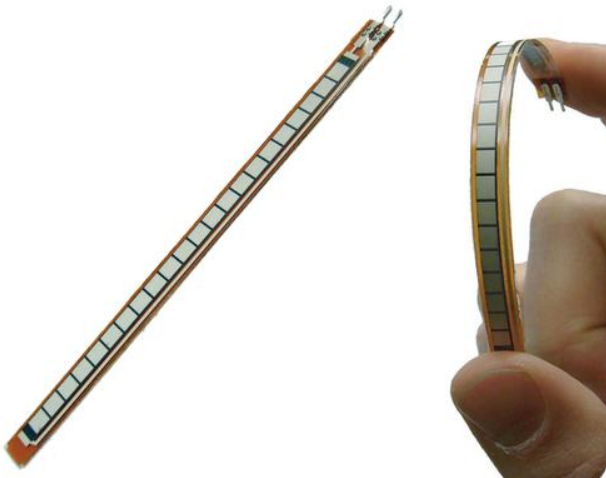
Así con este proyecto en un futuro poder llegar a las distintas áreas tanto en salud como en educación.

Objetivo

El objetivo es el de diseñar e implementar un prototipo utilizando sensores flex, un giroscopio y la placa arduino nano y mediante unity simular los gestos de una mano

Datos Técnicos

Sensores Flex

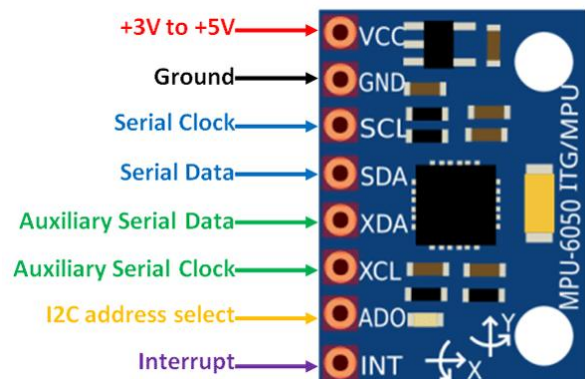


Los Sensores Flex son resistencias analógicas que trabajan como divisores de tensión analógica variable. Dentro de la flexión del sensor son elementos resistivos de carbono dentro de un sustrato flexible y delgado. (Más carbono significa menos resistencia). Cuando se dobla el sustrato del sensor produce una salida de resistencia en relación con el radio de curvatura. Con un sensor típico flex, una flexión de 0° dará la resistencia de 10K y una flexión de 90° dará entre 30 a 40 K ohmios.

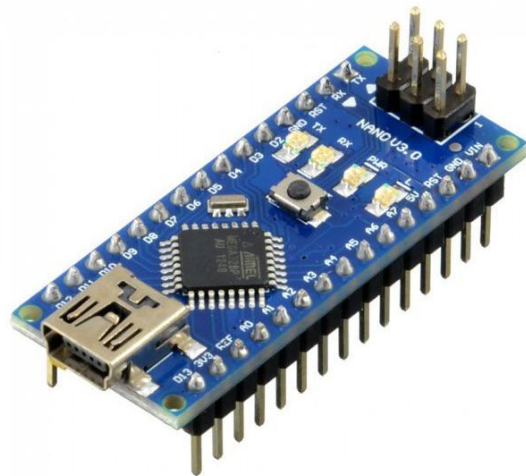
MPU-6050



El MPU-6050 es una unidad de medición inercial (IMU) de seis grados de libertad (6DOF) fabricado por Invensense, que combina un acelerómetro de 3 ejes y un giroscopio de 3 ejes. La comunicación puede realizarse tanto por SPI como por bus I2C, por lo que es sencillo obtener los datos medidos. La tensión de alimentación es de bajo voltaje entre 2.4 a 3.6V.



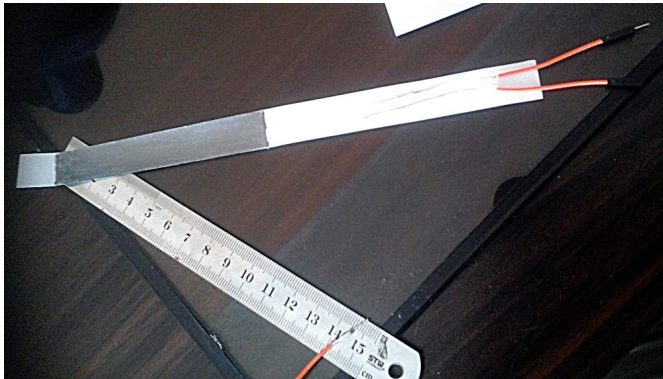
Arduino Nano



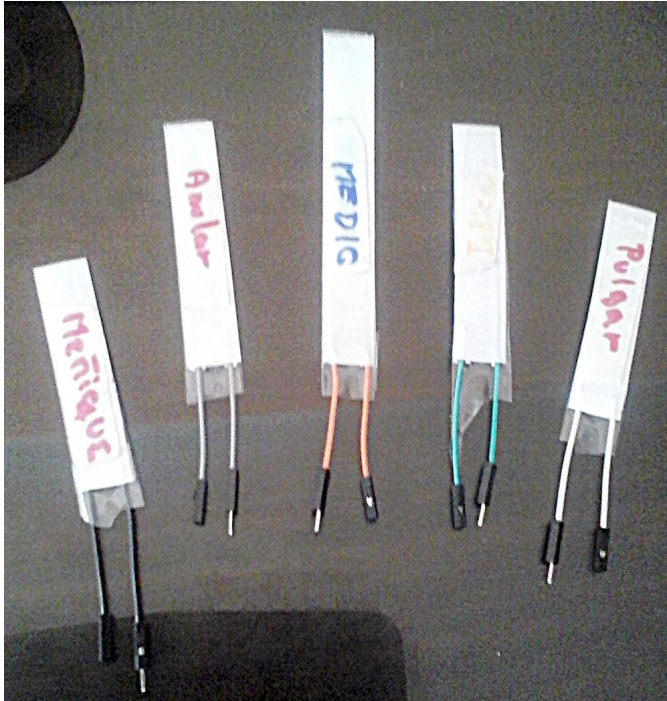
El Arduino Nano es una pequeña y completa placa basada en el ATmega328 (Arduino Nano 3.0) o el ATmega168 en sus versiones anteriores (Arduino Nano 2.x) que se usa conectándola a una protoboard. Tiene más o menos la misma funcionalidad que el Arduino Duemilanove, pero con una presentación diferente. No posee conector para alimentación externa, y funciona con un cable USB Mini-B.

Desarrollo del proyecto

Se construyeron los sensores flex caseros utilizando los siguientes materiales (papel bon, lápiz 5B, jumpers, cinta de aluminio, cinta masking, cinta diurex, regla, tijeras, pegamento), estos sensores nos permitirán simular los dedos de la mano.



Se realizó el mismo procedimiento para crear un sensor para cada dedo



Se realizó el montaje de los sensores en la estructura del brazo

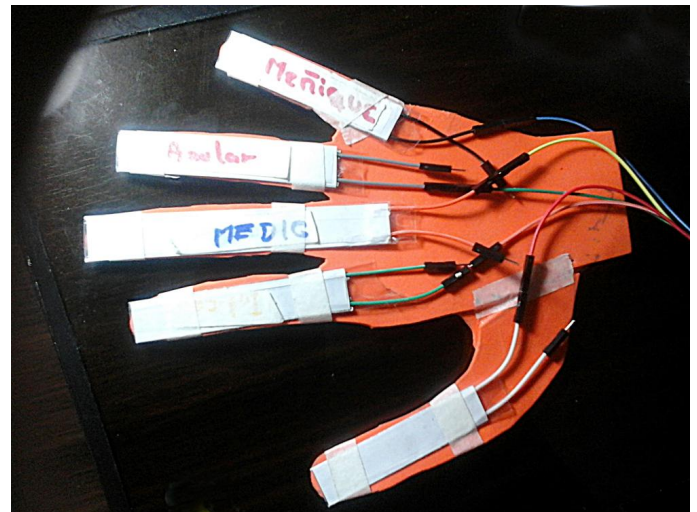
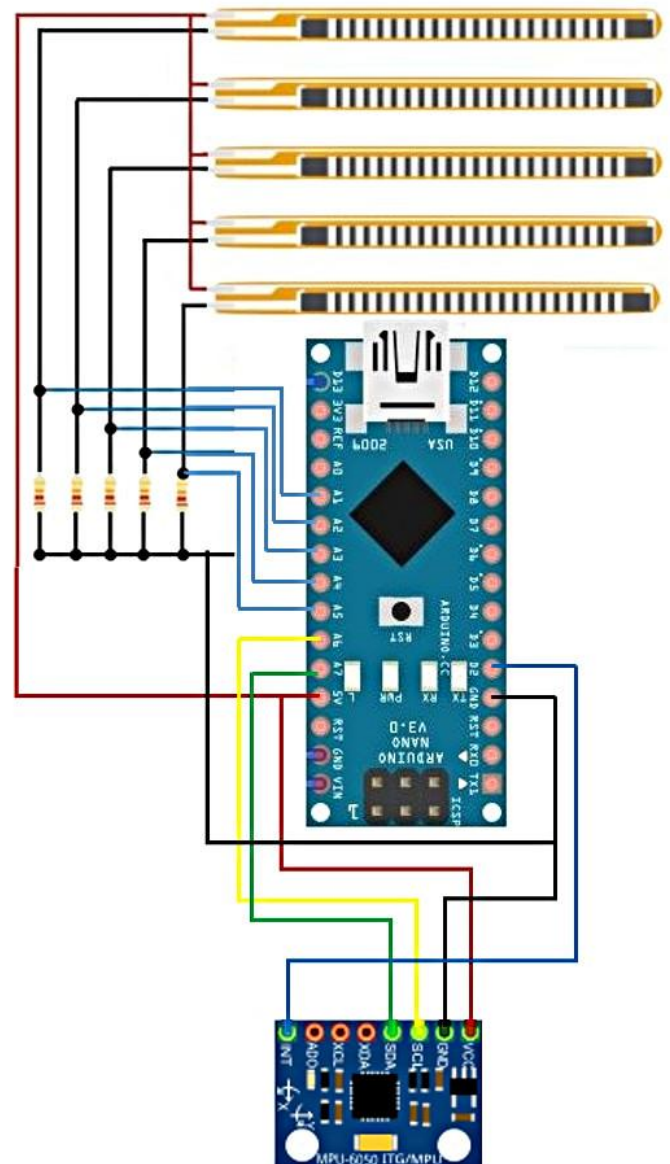
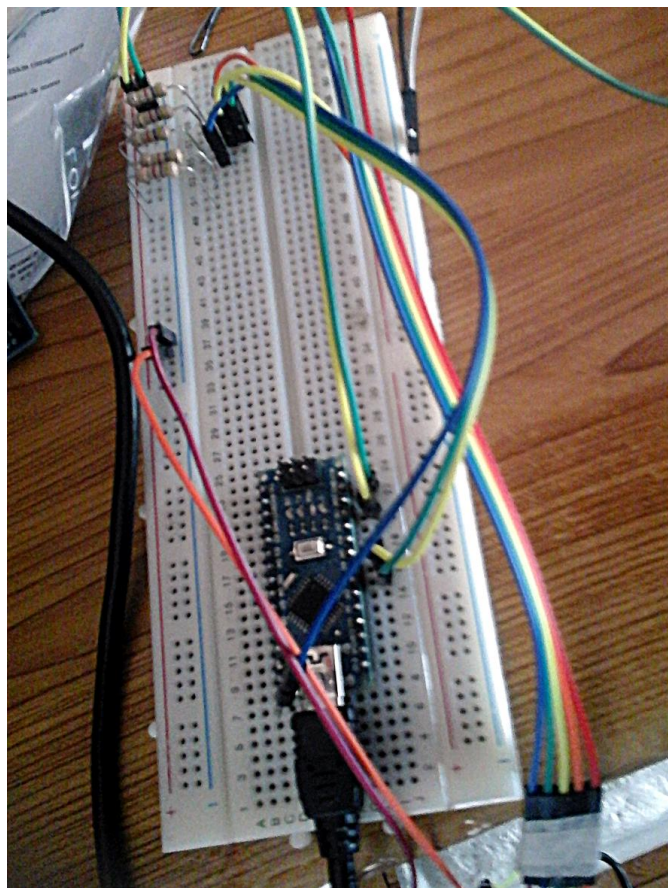
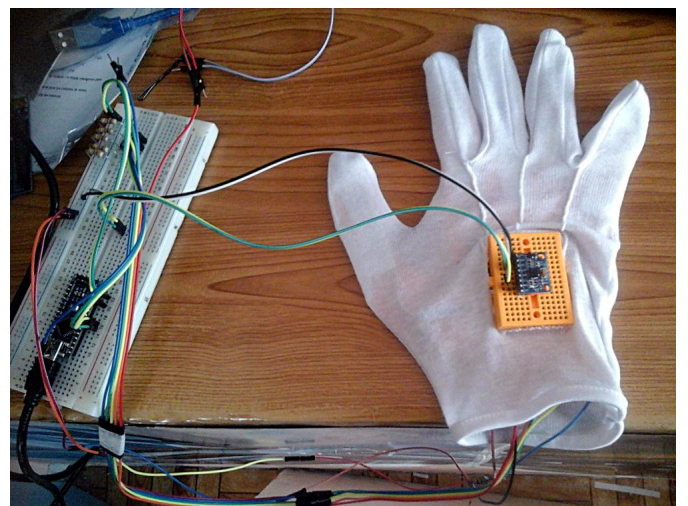
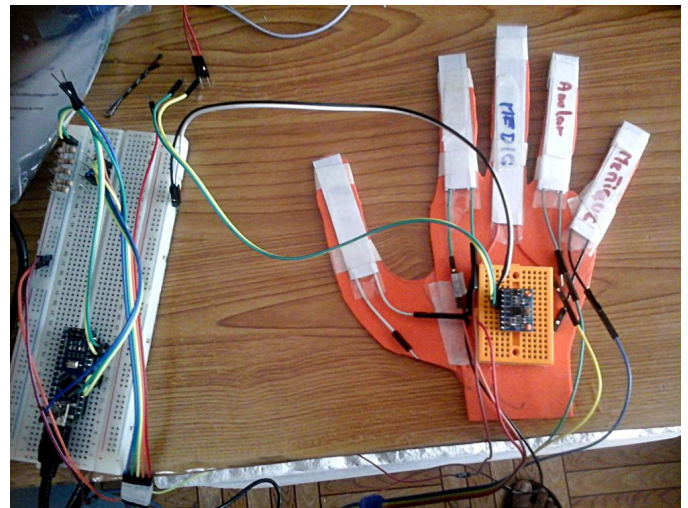
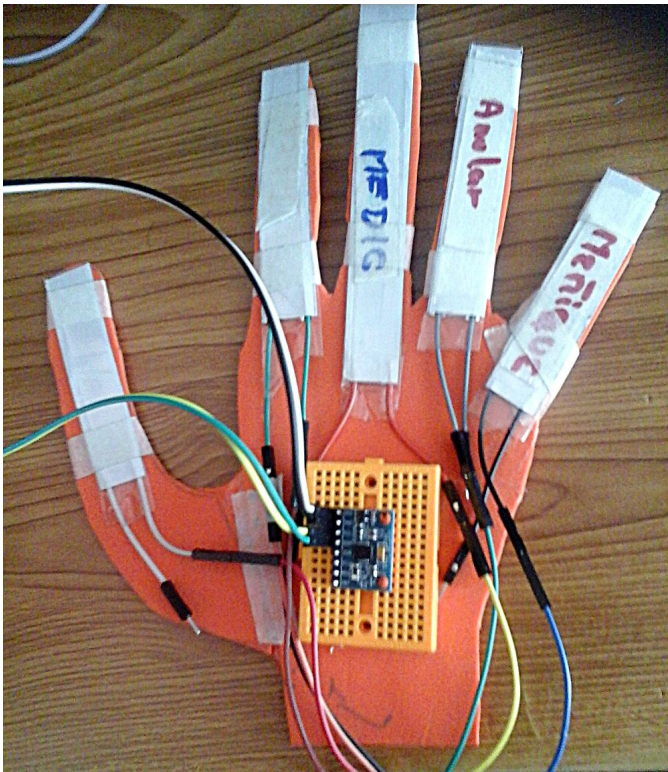


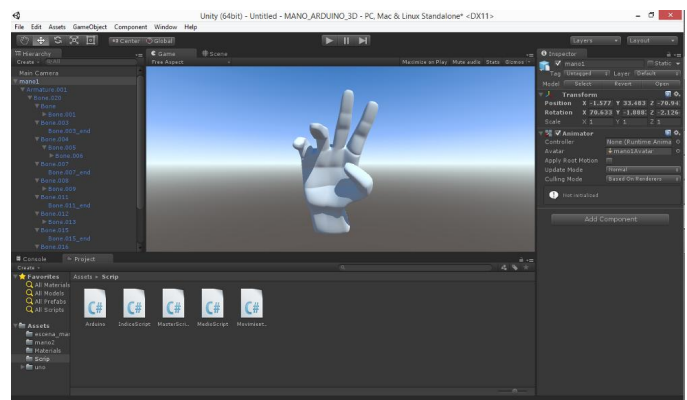
Diagrama electrónico del proyecto:

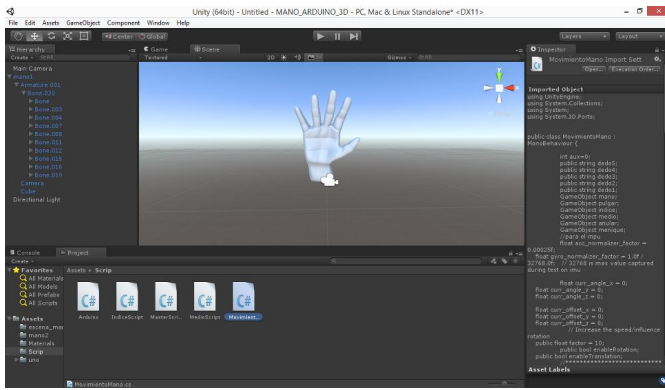


Se realizó el montaje del proyecto en el protoboard.



Se procedió a modelar una mano 3D utilizando la herramienta Blender, una vez termina el modelo se pasó a la Plataforma Unity para importar la mano 3D, continuando se utilizó la herramienta mecanic que pertenece a Unity para la creación de huesos de la mano 3D para darle un respectivo movimiento.





Código del Arduino:

```
int ledAZ=6;
int ledR=5;
int ledN=4;
int ledV=3;
int ledA=2;

int val5;
int val4;
int val3;
int val2;
int val1;

int dedo5=A7;
int dedo4=A6;
int dedo3=A3;
int dedo2=A2;
int dedo1=A1;
//para MPU*****
#include<Wire.h>
const int MPU=0x68; // I2C address of the MPU-6050
int16_t AcX,AcY,AcZ,Tmp,GyX,GyY,GyZ;
//para MPU*****
void setup() {
// put your setup code here, to run once:
Serial.begin(9600);
pinMode(ledAZ,OUTPUT);
pinMode(ledR,OUTPUT);
pinMode(ledN,OUTPUT);
pinMode(ledV,OUTPUT);
pinMode(ledA,OUTPUT);
//para MPU*****
Wire.begin();
Wire.beginTransmission(MPU);
Wire.write(0x6B); // PWR MGMT 1 register
Wire.write(0); // set to zero (wakes up the MPU-6050)
Wire.endTransmission(true);
//para MPU*****
}
void loop() {
//para dedo5 PULGAR
val5 = map(analogRead(dedo5), 750, 1023, 0, 340);
if(val5<100)
{
val5=400;
digitalWrite(ledAZ,HIGH);
//delay(100);
}
else
{
val5=-30;
digitalWrite(ledAZ,LOW);
}
//para dedo4 INDICE
val4 = map(analogRead(dedo4), 750, 1023, 0, 340);
if(val4<100)
{
val4=600;
digitalWrite(ledR,HIGH);
}
else
{
val4=0;
digitalWrite(ledR,LOW);
}
//para dedo3 MEDIO
val3 = map(analogRead(dedo3), 750, 1023, 0, 340);
if(val3<100)
{
val3=600;
digitalWrite(ledN,HIGH);
}
else
{
val3=0;
digitalWrite(ledN,LOW);
}
//para dedo2 ANULAR
val2 = map(analogRead(dedo2), 750, 1023, 0, 340);
if(val2<100)
{
val2=600;
digitalWrite(ledV,HIGH);
}
else
{
val2=0;
digitalWrite(ledV,LOW);
}
//para dedo1 MEÑIQUE
val1 = map(analogRead(dedo1), 750, 1023, 0, 340);
if(val1<100)
{
val1=600;
digitalWrite(ledA,HIGH);
}
else
{
val1=0;
digitalWrite(ledA,LOW);
}
// PARA EL MPU*****
Wire.beginTransmission(MPU);
Wire.write(0x3B); // starting with register 0x3B (ACCEL_XOUT_H)
Wire.endTransmission(false);
Wire.requestFrom(MPU,14,true); // request a total of 14 registers
AcX=Wire.read()<<8|Wire.read(); // 0x3B (ACCEL_XOUT_H) & 0x3C (ACCEL_XOUT_L)
AcY=Wire.read()<<8|Wire.read(); // 0x3D (ACCEL_YOUT_H) & 0x3E (ACCEL_YOUT_L)
AcZ=Wire.read()<<8|Wire.read(); // 0x3F (ACCEL_ZOUT_H) & 0x40 (ACCEL_ZOUT_L)
Tmp=Wire.read()<<8|Wire.read(); // 0x41 (TEMP_OUT_H) & 0x42 (TEMP_OUT_L)
GyX=Wire.read()<<8|Wire.read(); // 0x43 (GYRO_XOUT_H) & 0x44 (GYRO_XOUT_L)
GyY=Wire.read()<<8|Wire.read(); // 0x45 (GYRO_YOUT_H) & 0x46 (GYRO_YOUT_L)
GyZ=Wire.read()<<8|Wire.read(); // 0x47 (GYRO_ZOUT_H) & 0x48 (GYRO_ZOUT_L)
//*****
Serial.print(val5);Serial.print(";");Serial.print(val4);Serial.print(";");
Serial.print(val3);Serial.print(";");Serial.print(val2);Serial.print(";");
Serial.print(val1);Serial.print(";");Serial.print(AcX);Serial.print(";");
Serial.print(AcY);Serial.print(";");Serial.print(AcZ);Serial.print(";");
Serial.print(GyX);Serial.print(";");Serial.print(GyY);Serial.print(";");
Serial.print(GyZ);Serial.println("");
delay(100);
}
```

Código de Unity que permite el movimiento de la mano Recibiendo datos del Sensor MPU6050

```
using UnityEngine;
using System.Collections;
using System;
using System.IO.Ports;

public class MovimientoMano : MonoBehaviour {
    int aux=0;
    public string dedo5;
    public string dedo4;
    public string dedo3;
    public string dedo2;
    public string dedo1;
    GameObject mano;
    GameObject pulgar;
    GameObject indice;
    GameObject medio;
    GameObject anular;
    GameObject menique;
    //para el mpu
    float acc_normalizer_factor = 0.00025f;
    float gyro_normalizer_factor = 1.0f / 32768.0f;

    float curr_angle_x = 0;
    float curr_angle_y = 0;
    float curr_angle_z = 0;

    float curr_offset_x = 0;
    float curr_offset_y = 0;
    float curr_offset_z = 0;
    // Increase the speed/influence rotation
    public float factor = 10;
    public bool enableRotation;
    public bool enableTranslation;
    //*****

    SerialPort serial= new SerialPort("COM6",9600);
    void Start()
    {
        mano=GameObject.Find("mano1");
        pulgar=GameObject.Find("Dedo_Gordo");
        indice=GameObject.Find("Dedo_Indice");
        medio=GameObject.Find("Dedo_Medio");
        anular=GameObject.Find("Dedo_Anular");
        menique=GameObject.Find("Dedo_Menique");
        serial.Open ();
    }
    void Update(){
        //Debug.Log("se encontro el objeto "+medio.name);
        string datos=serial.ReadLine();
        string[] DatosFlex = datos.Split(';');
        dedo5=DatosFlex[0];
        dedo4=DatosFlex[1];
        dedo3=DatosFlex[2];
        dedo2=DatosFlex[3];
        dedo1=DatosFlex[4];
        ///para el MPU
        // recived string is like "accx;accy;accz;gyrox;gyroy;gyroz"
        //char splitChar = ';';
        //string[] dataRaw = dataString.Split(splitChar);
        // normalized accelerometer values
        float ax = Int32.Parse(DatosFlex[5]) * acc_normalizer_factor;
        float ay = Int32.Parse(DatosFlex[6]) * acc_normalizer_factor;
        float az = Int32.Parse(DatosFlex[7]) * acc_normalizer_factor;

        // normalized gyroscope values
        float gx = Int32.Parse(DatosFlex[8]) * gyro_normalizer_factor;
        float gy = Int32.Parse(DatosFlex[9]) * gyro_normalizer_factor;
        float gz = Int32.Parse(DatosFlex[10]) * gyro_normalizer_factor;

        // prevent
        if (Mathf.Abs(ax) - 1 < 0) ax = 0;
        if (Mathf.Abs(ay) - 1 < 0) ay = 0;
        if (Mathf.Abs(az) - 1 < 0) az = 0;

        curr_offset_x += ax;
        curr_offset_y += ay;
        curr_offset_z += 0; // The IMU module have value of z axis of 16600 caused by gravity

        // prevent little noise effect
        if (Mathf.Abs(gx) < 0.025f) gx = 0f;
        if (Mathf.Abs(gy) < 0.025f) gy = 0f;
        if (Mathf.Abs(gz) < 0.025f) gz = 0f;

        curr_angle_x += gx;
        curr_angle_y += gy;
        curr_angle_z += gz;

        transform.rotation = Quaternion.Euler(curr_angle_x * factor, -curr_angle_z * factor, curr_angle_y * factor);
    }
}
```


Código de Unity que permite el movimiento de los dedos Recibiendo datos de los sensores Flex.

```
using UnityEngine;
using System.Collections;
using System;
using System.IO.Ports;

public class MasterScript : MonoBehaviour {

    public string dedo5;
    public string dedo4;
    public string dedo3;
    public string dedo2;
    public string dedo1;

    //GameObject mano;
    GameObject pulgar;
    GameObject indice;
    GameObject medio;
    GameObject anular;
    GameObject menique;

    SerialPort serial= new SerialPort("COM6",9600);
    void Start()
    {
        //mano=GameObject.Find("manol");
        pulgar=GameObject.Find("Dedo_Gordo");
        indice=GameObject.Find("Dedo_Indice");
        medio=GameObject.Find("Dedo_Medio");
        anular=GameObject.Find("Dedo_Anular");
        menique=GameObject.Find("Dedo_Menique");

        serial.Open ();
    }
    void Update(){
        //Debug.Log("se encontro el objeto "+medio.name);
        string datos=serial.ReadLine();
        string[] DatosFlex = datos.Split(';');
        dedo5=DatosFlex[0];
        dedo4=DatosFlex[1];
        dedo3=DatosFlex[2];
        dedo2=DatosFlex[3];
        dedo1=DatosFlex[4];

        //transform.eulerAngles=new Vector3(int.Parse(DatosFlex[0]),0,-15);

        //transform.rotation = Quaternion.Euler(curr_angle_x * factor, -curr_angle_z * factor, curr_angle_y * factor);
        pulgar.transform.eulerAngles=new Vector3(int.Parse(DatosFlex[0]),300,int.Parse(DatosFlex[0]));
        indice.transform.eulerAngles=new Vector3(int.Parse(DatosFlex[1]),0,-10);
        medio.transform.eulerAngles=new Vector3(int.Parse(DatosFlex[2]),0f,0f);
        anular.transform.eulerAngles=new Vector3(int.Parse(DatosFlex[3]),0f,0f);
        menique.transform.eulerAngles=new Vector3(int.Parse(DatosFlex[4]),-10,0f);

    }
}
```

Impacto

Por otro lado, la realidad virtual es un mundo transversal en el que caben todo tipo de ideas e invenciones, y por tanto podremos utilizarlo en todas las materias existentes de la etapa educativa. La clave será, como tantas otras veces, el material que las compañías especializadas estén dispuestas a crear.

Elaborado por:



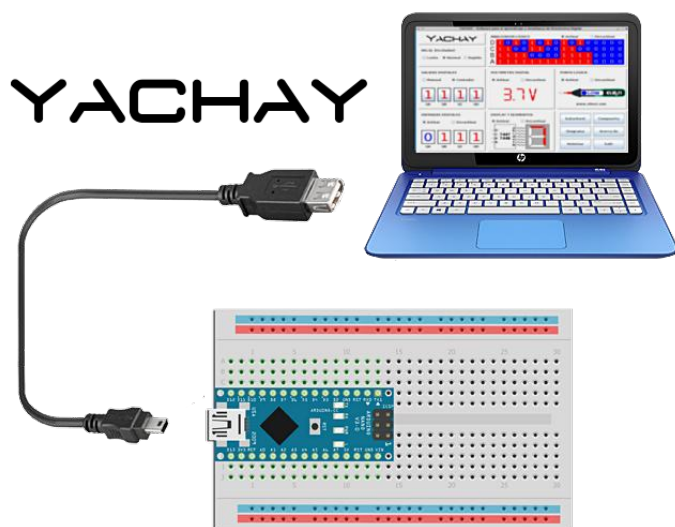
Cordova Conde Jose Luis

Nacido en la ciudad de La Paz Bolivia, estudiante de último año de la Carrera de Informática Universidad Mayor de San Andrés (UMSA), Técnico Superior Analista de Sistemas Informático, fui supervisor de toma de pruebas de la segunda Olimpiada Científica Estudiantil Plurinacional Boliviana(OCEPB) en el área de Informática, Instructor de Programación para (OCEPB), integrante de la comunidad Arduino Open Source UMSA.

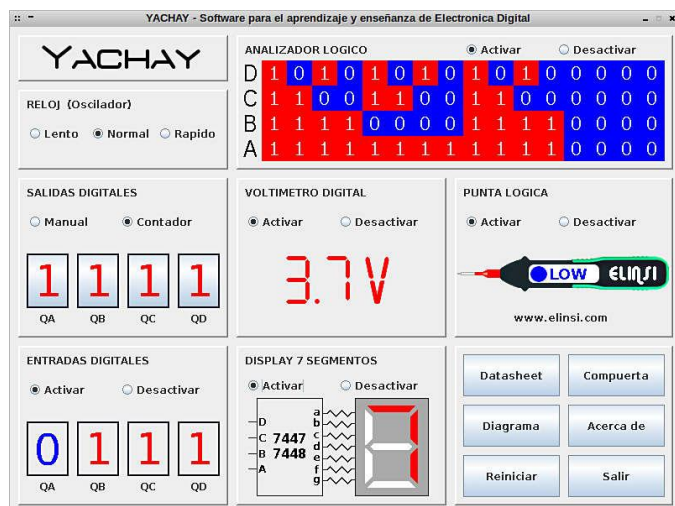
YACHAY - Herramienta para el aprendizaje y enseñanza de la Electrónica Digital

YACHAY es un sistema de software y hardware diseñado para ser una herramienta para el aprendizaje y la enseñanza de la electrónica digital, diseñado y programado por Osman R. Condori Guevara y se puede descargar de forma gratuita de la pagina oficial de ELINSI:

http://elinsi.com/p_yachay.php

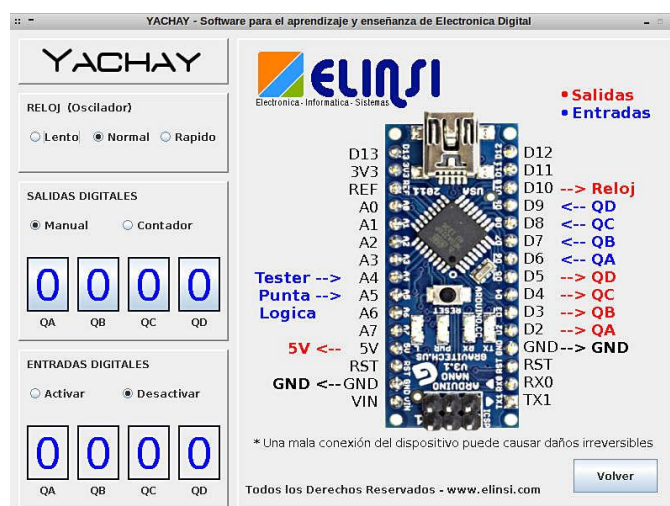


Yachay es una palabra quechua que significa “aprender”, el quechua es una de las lenguas oficiales de Bolivia.



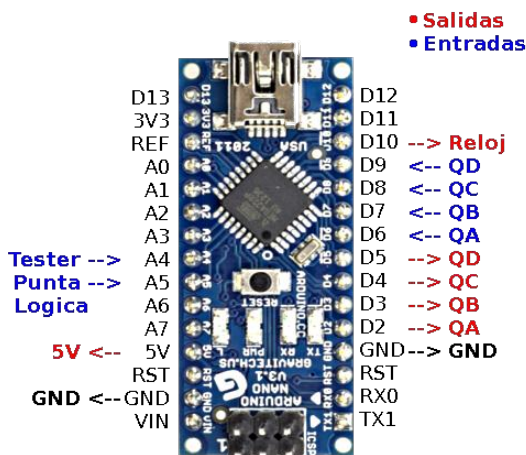
YACHAY nos permite contar con todos los instrumentos y herramientas necesarias para el aprendizaje y enseñanza de la electrónica digital, la aplicación se ejecuta en una computadora y a través de un cable usb se conecta al Arduino Nano, a través de la interfaz de usuario se puede controlar el estado de las salidas digitales y también nos permite visualizar el estado lógico de las entradas digitales de cualquier circuito digital, el sistema cuenta con un analizador lógico que nos permite visualizar el cambio de los valores lógicos de cualquier circuito digital, tiene un voltímetro para poder medir/verificar los voltajes existentes en diferentes etapas de un circuito, también cuenta con una punta lógica que no permitirá ver el estado lógico presente en cualquier punto del circuito ("0" lógico, "1" lógico o indeterminado), se puede aprovechar la alimentación del Arduino (5V) para alimentar nuestros proyectos que se armen en el protoboard.

Hardware del sistema



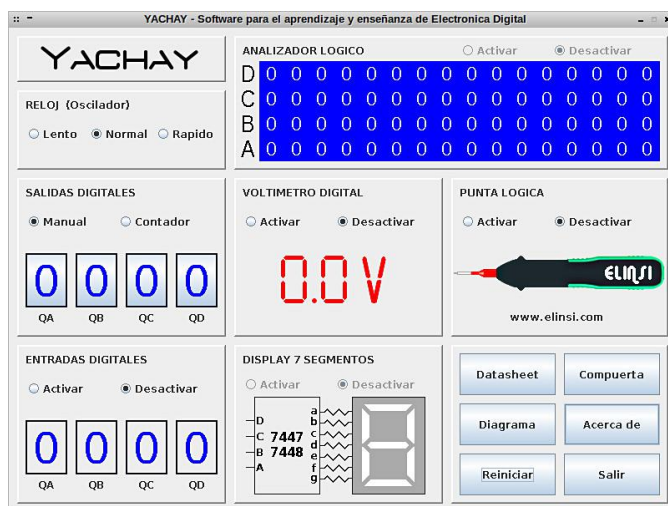
El hardware esta formado por la placa Arduino Nano, pero puede ser implementado en cualquier modelo que sea compatible con éste

Arduino como el Arduino UNO, mega, leonardo, etc. El arduino es el encargado de establecer la comunicación de las entradas y salidas digitales del sistema con el software del PC.



En la figura de arriba se observa los pines utilizados del Arduino, todos los pines que están marcados de color azul son pines de entrada y los pines marcados de color rojo son pines de salida, se debe tener mucho cuidado a la hora de hacer las conexiones.

Software del sistema

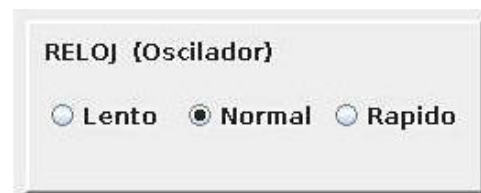


El software fue programado en java y no necesita instalarse, el ejecutable es portable y puede correr en sistemas operativos Windows y cualquier distribución GNU/Linux (Ubuntu, Debian, etc.), ésta interfaz permite al usuario

interactuar con la herramienta YACHAY y a la vez el software establece la comunicación con el arduino para la comunicación con el hardware.

Partes del Software

Reloj (Oscilador)



El sistema cuenta con un reloj (pin 10 del Arduino) que genera una señal onda cuadrada con tres frecuencias distintas, a través del software se realiza la selección de la frecuencia de trabajo y así obtener una señal de onda cuadas (señal de reloj) para nuestros proyectos digitales.

Salidas Digitales (Q1, Q2, Q3 y Q4)



El sistema cuenta con cuatro salidas digitales (pines D2 al D5 del Arduino) donde a través del software se puede controlar los estados lógicos de la salida y poderlos utilizar como entradas digitales de nuestro circuito digital, tiene dos formas de trabajo que son manual y contador

Manual: Las salidas son controladas de forma manual, a través del mouse podemos cambiar

los estados lógicos de cada uno de las salidas del sistema.

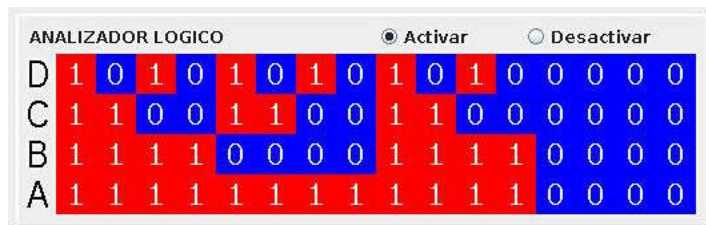
Contador: Las salidas del sistema son controladas automaticamente, el sistema comienza a generar un conteo digital (las salidas logicas cambian de estado en cada pulso del reloj del sistema)

Entradas Digitales (Q1, Q2, Q3 y Q4):



El sistema cuenta con cuatros entradas digitales (pines D6 al D9 del Arduino) que a través del software se puede visualizar el estado lógico de cada uno de los pines digitales que esten conectados en nuestro circuito digital.

Analizador Lógico:



El sistema cuenta con un analizador lógico de cuatro canales (pines D6 al D9 del Arduino), un analizador logico es un instrumento de medida que captura los datos de un circuito digital y los muestra en pantalla para su posterior análisis.

Punta Lógica



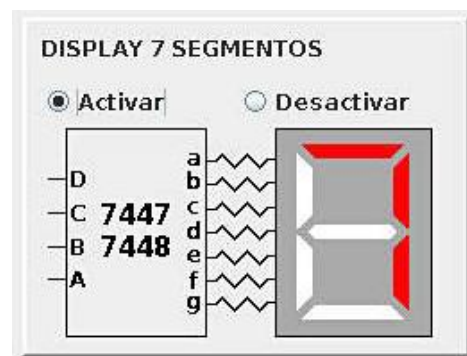
El sistema cuenta con un punta lógica (pin A5 del Arduino) que es un instrumento utilizado en la electrónica digital para determinar el estado lógico en los distintos puntos de un circuito, la punta lógica nos indicará si se encuentra en el estado lógico "1", "0" o "indeterminado"

Voltimetro Digital



El sistema cuenta con un voltimetro digital (pin A4 del Arduino) que nos permite medir voltajes en distintos puntos del circuito, **el voltaje máximo de medición es de 5V.**

Decodificador de 7 Segmentos



El sistema cuenta con decodificador de BCD a 7 Segmentos (pines D6 al D9 del Arduino), el valor de conteo binario BCD que se recibe a la entrada del sistema es visualizado en un display de 7 segmentos en la interfaz gráfica.

Diagrama de compuertas lógicas, símbolos, tablas de verdad y diagrama de conexión



A través de los botones se puede acceder a los datasheet de las compuertas lógicas mas utilizadas, símbolo de las compuertas lógicas, tablas de verdad, funciones lógicas y el diagrama de conexionado del Arduino para poder utilizar el sistema.

YACHAY - Software para el aprendizaje y enseñanza de Electrónica Digital

RELOJ (Oscilador)
☐ Lento ☒ Normal ☐ Rapido

SALIDAS DIGITALES
☒ Manual ☐ Contador

ENTRADAS DIGITALES
☐ Activar ☒ Desactivar

Función	Ecuación lógica	Símbolos		Tabla de verdad															
		Norma MIL	Norma IEC																
OR	$S = A + B$			<table border="1"> <tr><td>A</td><td>B</td><td>S</td></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	A	B	S	0	0	0	0	1	1	1	0	1	1	1	1
A	B	S																	
0	0	0																	
0	1	1																	
1	0	1																	
1	1	1																	
AND	$S = A \cdot B$			<table border="1"> <tr><td>A</td><td>B</td><td>S</td></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	A	B	S	0	0	0	0	1	0	1	0	0	1	1	1
A	B	S																	
0	0	0																	
0	1	0																	
1	0	0																	
1	1	1																	
NOT	$S = \bar{A}$			<table border="1"> <tr><td>A</td><td>S</td></tr> <tr><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td></tr> </table>	A	S	0	1	1	0									
A	S																		
0	1																		
1	0																		
NOR (OR+NOT)	$S = \overline{A + B}$ $S = \bar{A} \cdot \bar{B}$			<table border="1"> <tr><td>A</td><td>B</td><td>S</td></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	A	B	S	0	0	1	0	1	0	1	0	0	1	1	0
A	B	S																	
0	0	1																	
0	1	0																	
1	0	0																	
1	1	0																	
NAND (AND+NOT)	$S = \overline{A \cdot B}$ $S = \bar{A} + \bar{B}$			<table border="1"> <tr><td>A</td><td>B</td><td>S</td></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	A	B	S	0	0	1	0	1	1	1	0	1	1	1	0
A	B	S																	
0	0	1																	
0	1	1																	
1	0	1																	
1	1	0																	
EXOR	$S = A \oplus B = \bar{A}B + A\bar{B}$			<table border="1"> <tr><td>A</td><td>B</td><td>S</td></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	A	B	S	0	0	0	0	1	1	1	0	1	1	1	0
A	B	S																	
0	0	0																	
0	1	1																	
1	0	1																	
1	1	0																	

Todos los Derechos Reservados - www.elinsi.com

YACHAY - Software para el aprendizaje y enseñanza de Electrónica Digital

RELOJ (Oscilador)
☐ Lento ☒ Normal ☐ Rapido

SALIDAS DIGITALES
☒ Manual ☐ Contador

ENTRADAS DIGITALES
☐ Activar ☒ Desactivar

ELINSI
 Electrónica - Informática - Sistemas

• Salidas
 • Entradas

D13 3V3
 REF
 A0
 A1
 A2
 A3
 A4
 A5
 A6
 A7
 5V
 GND

D12 D11
 D10 --> Reloj
 D9 --> QD
 D8 --> QC
 D7 --> QB
 D6 --> QA
 D5 --> QD
 D4 --> QC
 D3 --> QB
 D2 --> QA
 GND--> GND
 RST
 RX0
 TX1

Tester -->
 Punta -->
 Logica

* Una mala conexión del dispositivo puede causar daños irreversibles

Todos los Derechos Reservados - www.elinsi.com

YACHAY - Software para el aprendizaje y enseñanza de Electrónica Digital

RELOJ (Oscilador)
☐ Lento ☒ Normal ☐ Rapido

SALIDAS DIGITALES
☒ Manual ☐ Contador

ENTRADAS DIGITALES
☐ Activar ☒ Desactivar

ELINSI
 Electrónica - Informática - Sistemas

NAND : 74LS00
 C.I. : 74LS00

NOR : 74LS02
 NOT : 74LS04

AND : 74LS08
 OR : 74LS32

XOR : 74LS86
 XNOR : 74LS266

Todos los Derechos Reservados - www.elinsi.com

YACHAY - Software para el aprendizaje y enseñanza de Electrónica Digital

RELOJ (Oscilador)
☐ Lento ☒ Normal ☐ Rapido

SALIDAS DIGITALES
☒ Manual ☐ Contador

ENTRADAS DIGITALES
☐ Activar ☒ Desactivar

ELINSI
 Electrónica - Informática - Sistemas

YACHAY es una plataforma de Software y Hardware diseñado y programado por la empresa ELINSI como una herramienta para el aprendizaje y la enseñanza de la Electrónica Digital.

* La empresa ELINSI no se hace responsable por daños producidos por el mal uso y manejo de la plataforma YACHAY

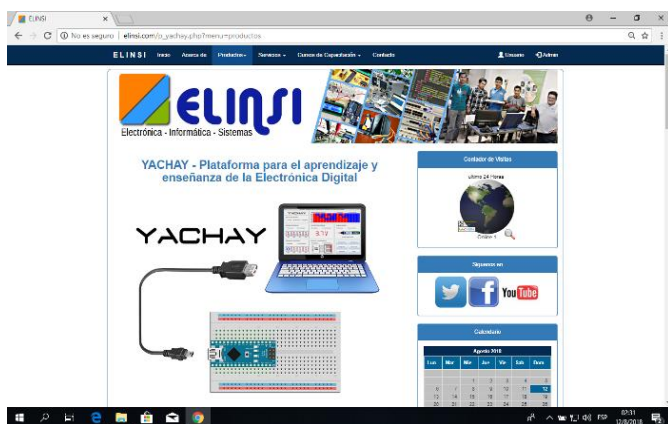
Programado por Osman R. Condori Guevara
 E-mail: osman@elinsi.com
 Celular: 79390520

Todos los Derechos Reservados - www.elinsi.com

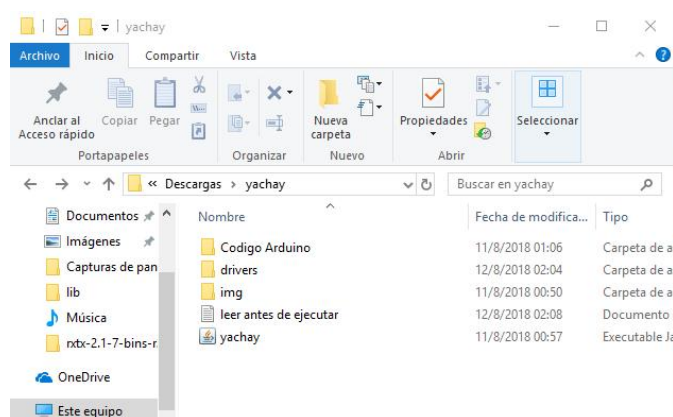
Instalación de Yachay en computadora con sistema operativo Windows

A continuación se describe el procedimiento de instalación de yachay en computadoras que tienen el sistema operativo Windows, el procedimiento es similar para otros sistemas operativos, la única diferencia es en la instalación del driver.

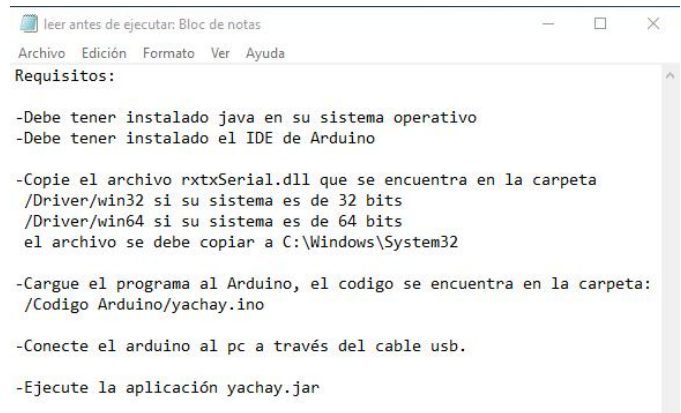
Descargamos el archivo comprimido yachay.zip de la pagina oficial de [ELINSI](http://elinsi.com):
http://elinsi.com/p_yachay.php



Descomprimir el archivo yachay.zip, se creará una carpeta con el nombre "yachay", ingrese a la carpeta y verá el siguiente contenido:



Los requisitos se encuentran en el archivo de texto: "leer antes de ejecutar" abrir con cualquier editor de texto



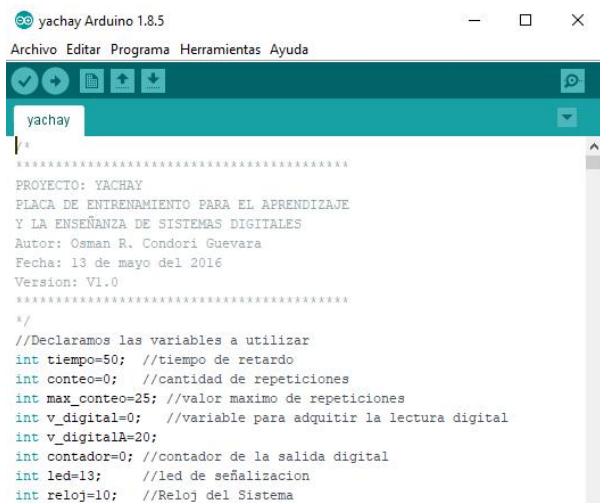
Si no tiene instalado java en su sistema operativo lo puede descargar de:
https://www.java.com/es/download/windows_manual.jsp

Si no tiene instalado el IDE de arduino lo puede descargar de:
<https://www.arduino.cc/en/Main/Software>

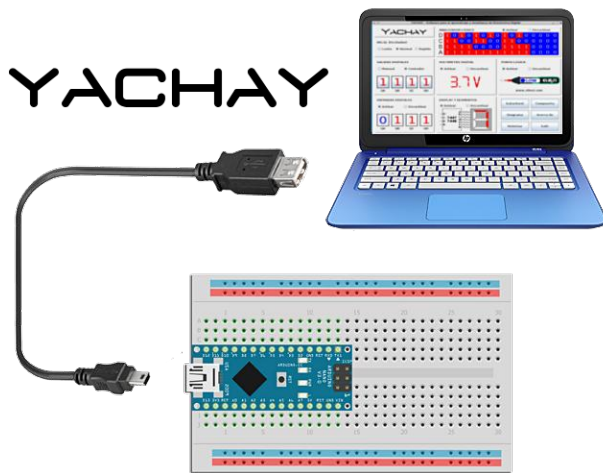
Copie el archivo rtxSerial.dll a:
C:\Windows\System32
el archivo se encuentra en la carpeta /Driver/win32 si su sistema es de 32 bits y /Driver/win64 si su sistema es de 64 bits

Nombre	Fecha de modifica...
rtxSerial.dll	4/2/2009 21:12

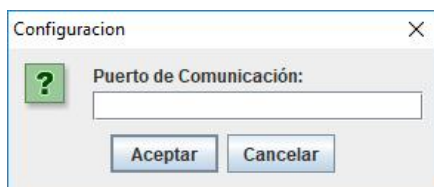
Cargue el código al Arduino, el archivo del código se encuentra en la carpeta: /Codigo Arduino/yachay.ino



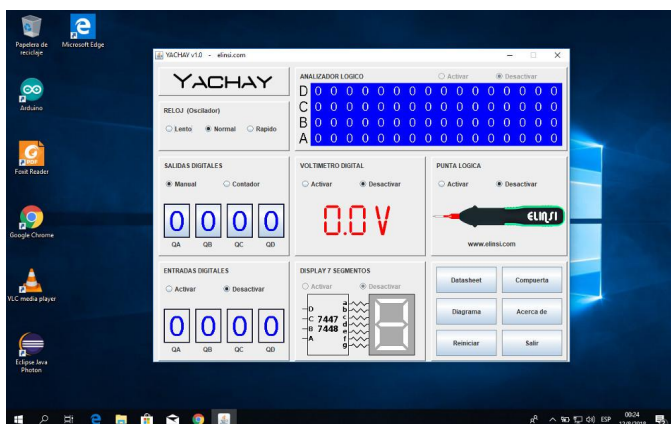
Conecte el Arduino al computador a través del cable USB



Ejecute el ejecutable "yachay.jar" (no necesita instalarse), le va a solicitar que escriba el puerto de comunicación Serial que se le asignado al Arduino, Ejm: COM4



Se ejecutara la aplicación YACHAY y el sistema se encontrará listo para su funcionamiento



Si tiene algun error a la hora de instalar o encuentra alguna falla en el sistema, pongase en contacto a través del correo:

osman@elinsi.com

Tanto la empresa ELINSI como mi persona no se hace responsable por cualquier daño que pueda existir por el mal uso del sistema, por mas de dos años lo vengo usando con mis alumnos y no e tenido ningun tipo de problemas en el funcionamiento.

Elaborado por:



Osman R. Condori Guevara

Nacido en la ciudad de Cochabamba - Bolivia, estudió Ingeniería Electrónica en la Universidad Mayor de San Simón (UMSS) y Técnico Superior en Electrónica en la Universidad de San Francisco Xavier de Chuquisaca (UMRPSFXCH), propietario de la Empresa de servicios y capacitación técnica en Electrónica Informática y Sistemas [ELINSI](http://www.elinsi.com), osman@elinsi.com

